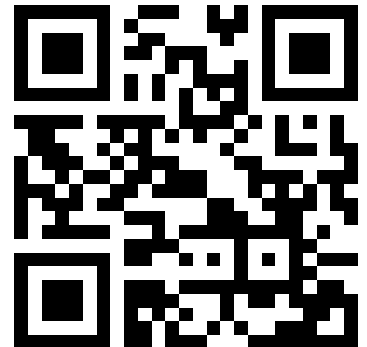


Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>



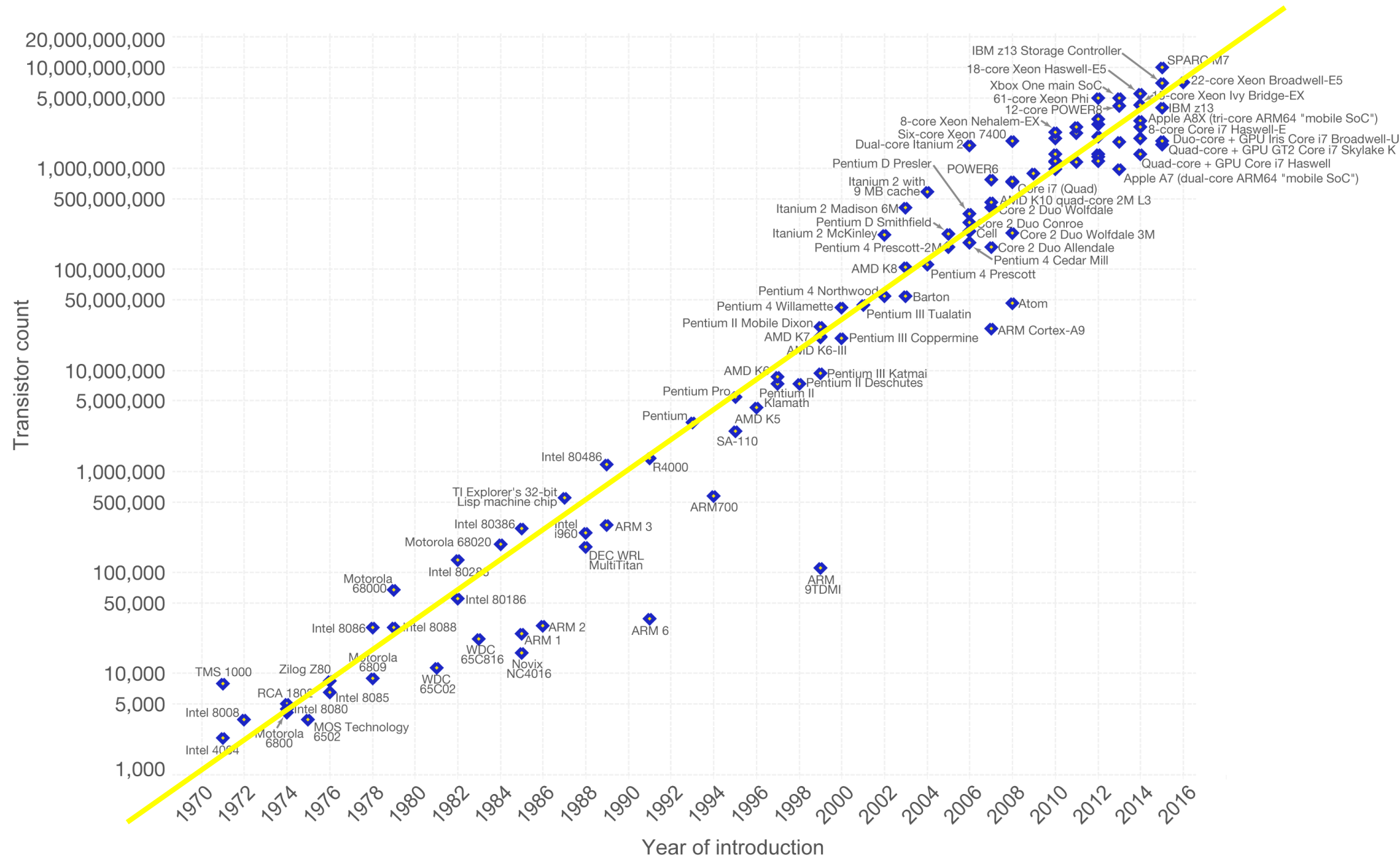
Embedded Systems

Flight Management System For Gliders



Moore's Law – The number of transistors on integrated circuit chips (1971-2016)

Moore's law describes the empirical regularity that the number of transistors on integrated circuits doubles approximately every two years. This advancement is important as other aspects of technological progress – such as processing speed or the price of electronic products – are strongly linked to Moore's law.

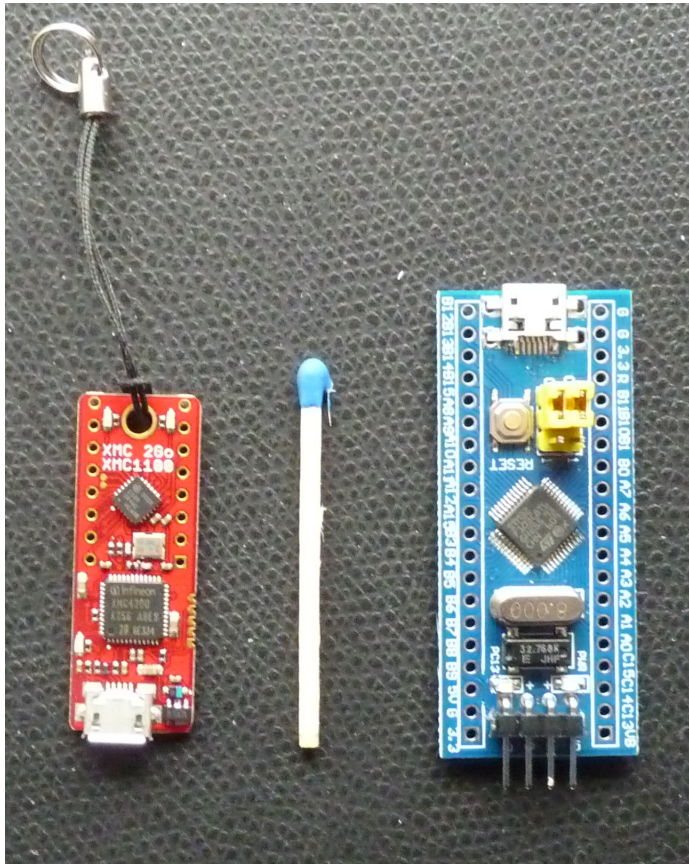


Data source: Wikipedia (https://en.wikipedia.org/wiki/Transistor_count)

The data visualization is available at [OurWorldinData.org](https://ourworldindata.org). There you find more visualizations and research on this topic.

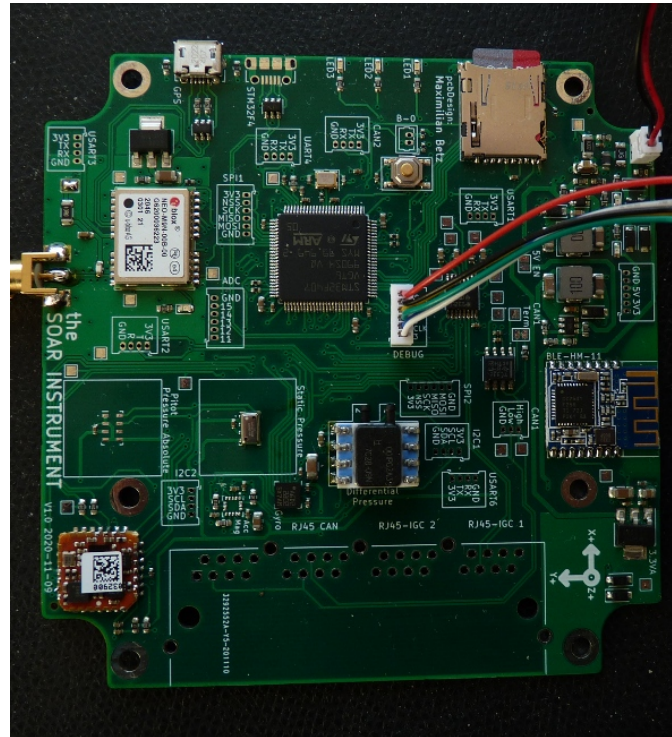
Licensed under [CC-BY-SA](https://creativecommons.org/licenses/by-sa/4.0/) by the author Max Roser.

Cortex M Micro-Controller Family



CM0

CM3



CM4



CM7

ARM Family of Microcontrollers

Cortex M RISC Product Line

Cortex M0

Von - Neumann Architecture

Very low Gate-Count

Ultra-Low-Power down to 3 μ W / MHz

Cortex M3

Optimized Instruction Set “Thumb2”

I/O Subsystem, Programming (CMSIS)

Interrupt Controller (NVIC)

Cortex M4

Memory Protection Unit (MPU)

Digital Signal Processing (DSP)

Floating Point Computation Unit (FPU 32 bit)

Cortex M7

Data and Instruction Cache

6-Stage Pipeline w. Branch-Prediction

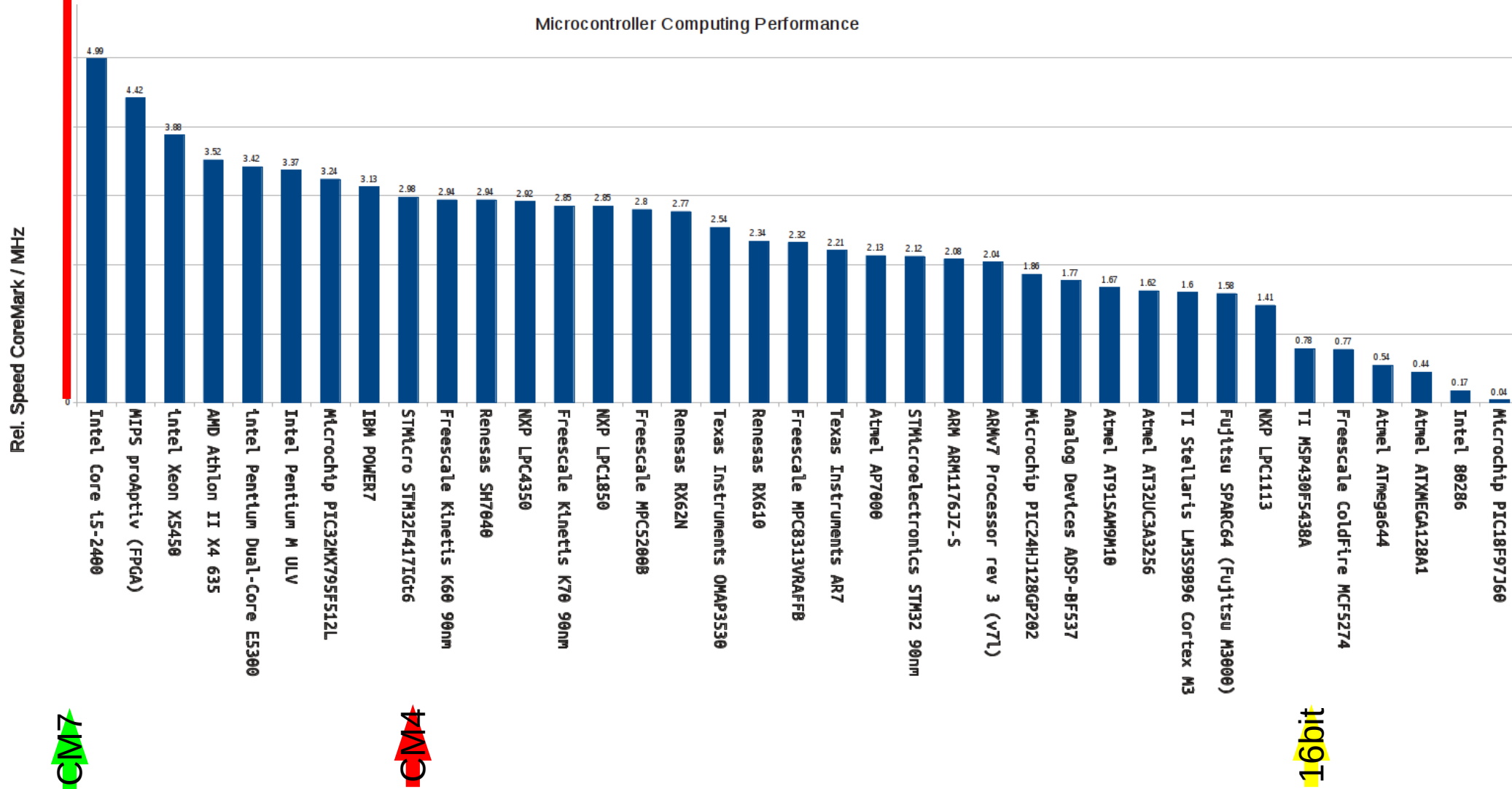
64-bit Data Paths

64-bit Floating Point Unit

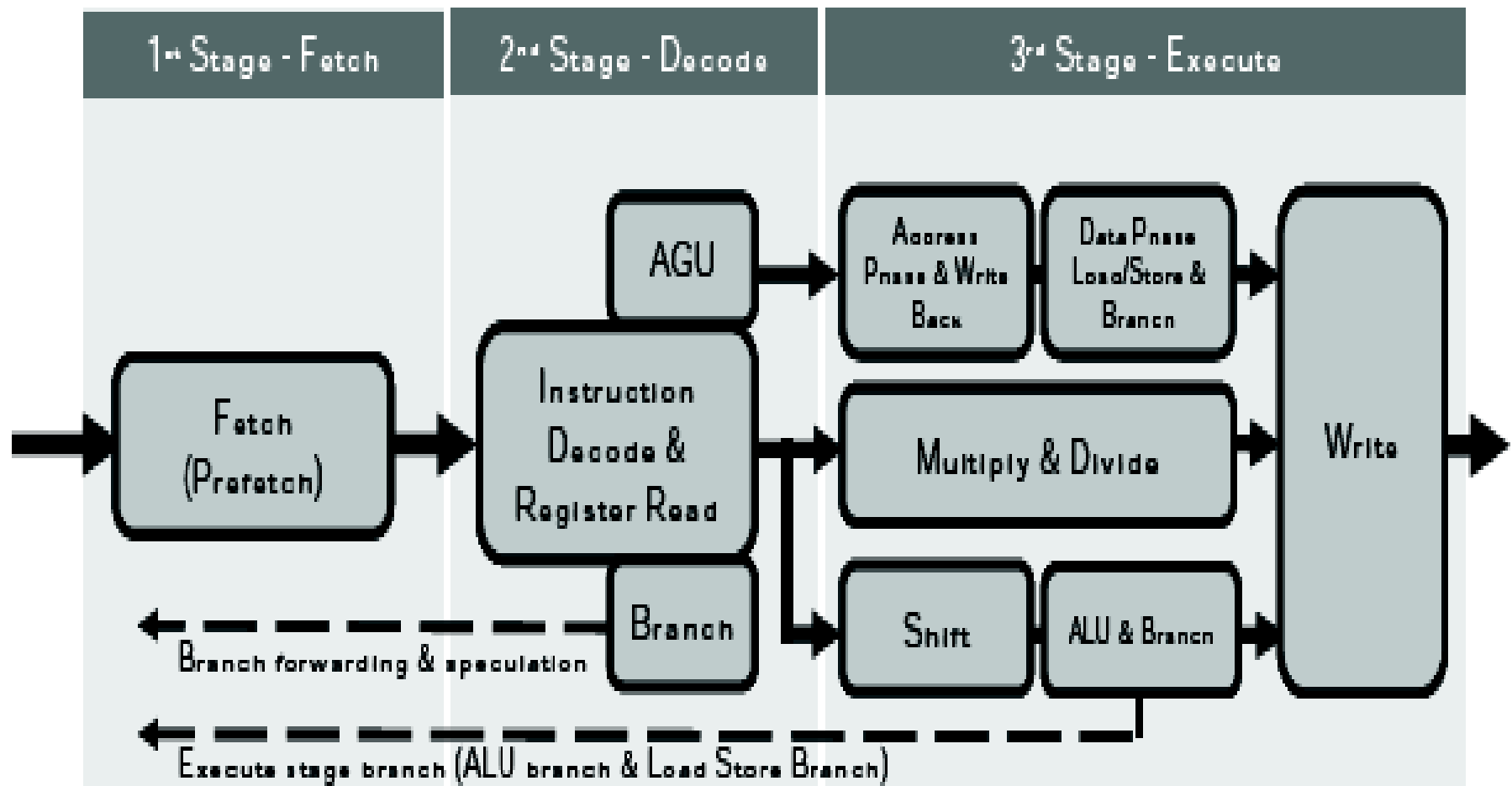
Architectural Performance

8.96 STM32H745

Coremark Units / MHz

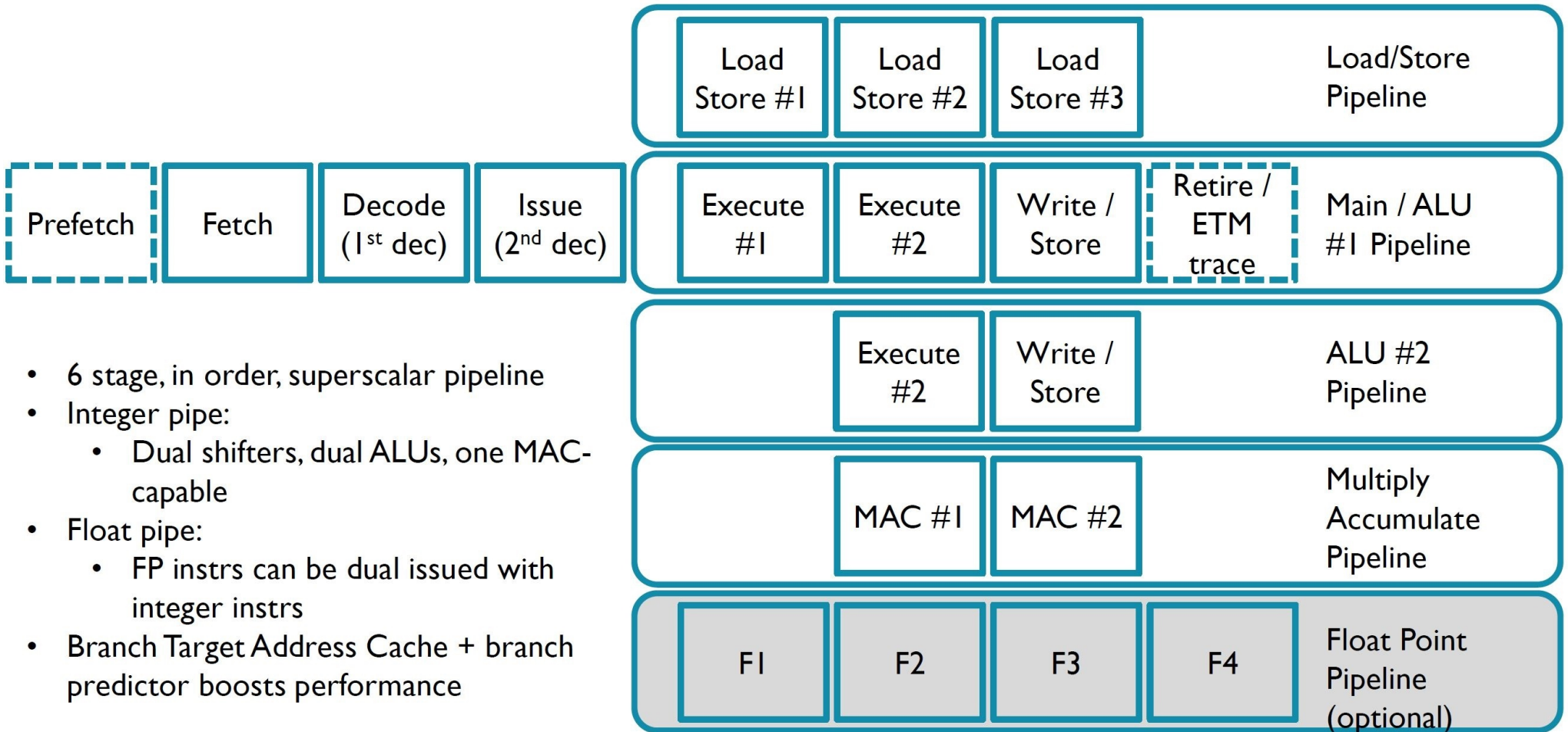


3 Stage Pipeline



Remark: The CM0(+) has a 2 stage pipeline only

Cortex-M7 Pipeline



- 6 stage, in order, superscalar pipeline
- Integer pipe:
 - Dual shifters, dual ALUs, one MAC-capable
- Float pipe:
 - FP instrs can be dual issued with integer instrs
- Branch Target Address Cache + branch predictor boosts performance

1

2

3

4

5

6

7

VABS	VADD	VCMP	VCMPPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			

Cortex-M4 FPU

PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLSd	SMLSd	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB

ADC	ADD	ADR	AND	ASR	B
CLZ	BFC	BFi	BIC	CDP	CLREX
CBNZ	CBZ	CMN	CMP	DBG	LDC
LDMIA	LDMDb	LDR	LDRB	LDRBT	LDRD
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL
LSR	MCRR	MLS	MLA	MOV	MOVT
MRC	MRRC	MUL	MVN	NOP	ORN
ORR	PLD	PLDW	PLI	POP	PUSH
RBIT	REV	REV16	REVSH	ROR	RRX

BKPT	BLX	ADC	ADD	ADR
BX	CPS	AND	ASR	B
DMB		BL		BIC
DSB	CMN	CMP		EOR
ISB	LDR	LDRB	LDM	
MRS	LDRH	LDRSB	LDRSH	
MSR	LSL	LSR	MOV	
NOP	REV	MUL	MVN	ORR
REV16	REVSH	POP	PUSH	ROR
SEV	SXTB	RSB	SBC	STM
SXTH	UXTB	STR	STRB	STRH
UXTH	WFE	SUB	SVC	TST
WFI	YIELD			

Cortex-M0/M0+/M1

RSB	SBC	SBFX
SDIV	SEV	SMLAL
SMULL	SSAT	STC
STMIA	STMDb	STR
STRB	STRBT	STRD
STREX	STREXB	STREXH
STRH	STRHT	STRT
SUB	SXTB	SXTH
TBB	TBH	TEQ
TST	UBFX	UDIV
UMLAL	UMULL	USAT
UXTB	UXTH	WFE
WFI	YIELD	IT

Cortex-M3

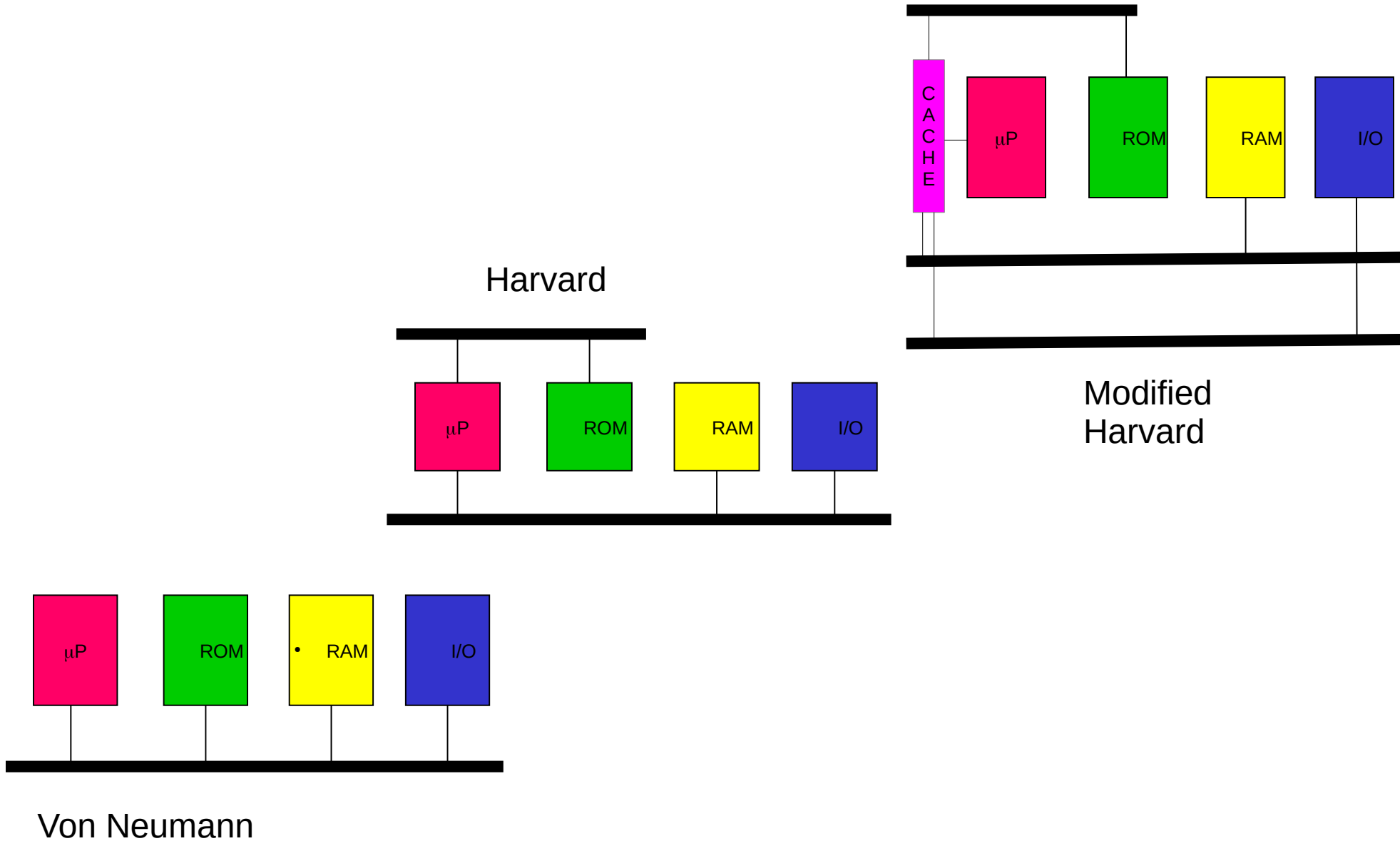
SMULBT	SMULTT
SMULTB	SMULWT
SMULWB	SMUSD
SSAT16	SSAX
SSUB16	SSUB8
SXTAB	SXTAB16
SXTAH	SXTB16
UADD16	UADD8
UASX	UHADD16
UHADD8	UHASX
UHSAX	UHSUB16
UHSUB8	UMAAL
UQADD16	UQADD8
UQASX	UQSAX
UQSUB16	UQSUB8
USAD8	USADA8
USAT16	USAX
USUB16	USUB8
UXTAB	UXTAB16
UXTAH	UXTB16

Cortex-M4

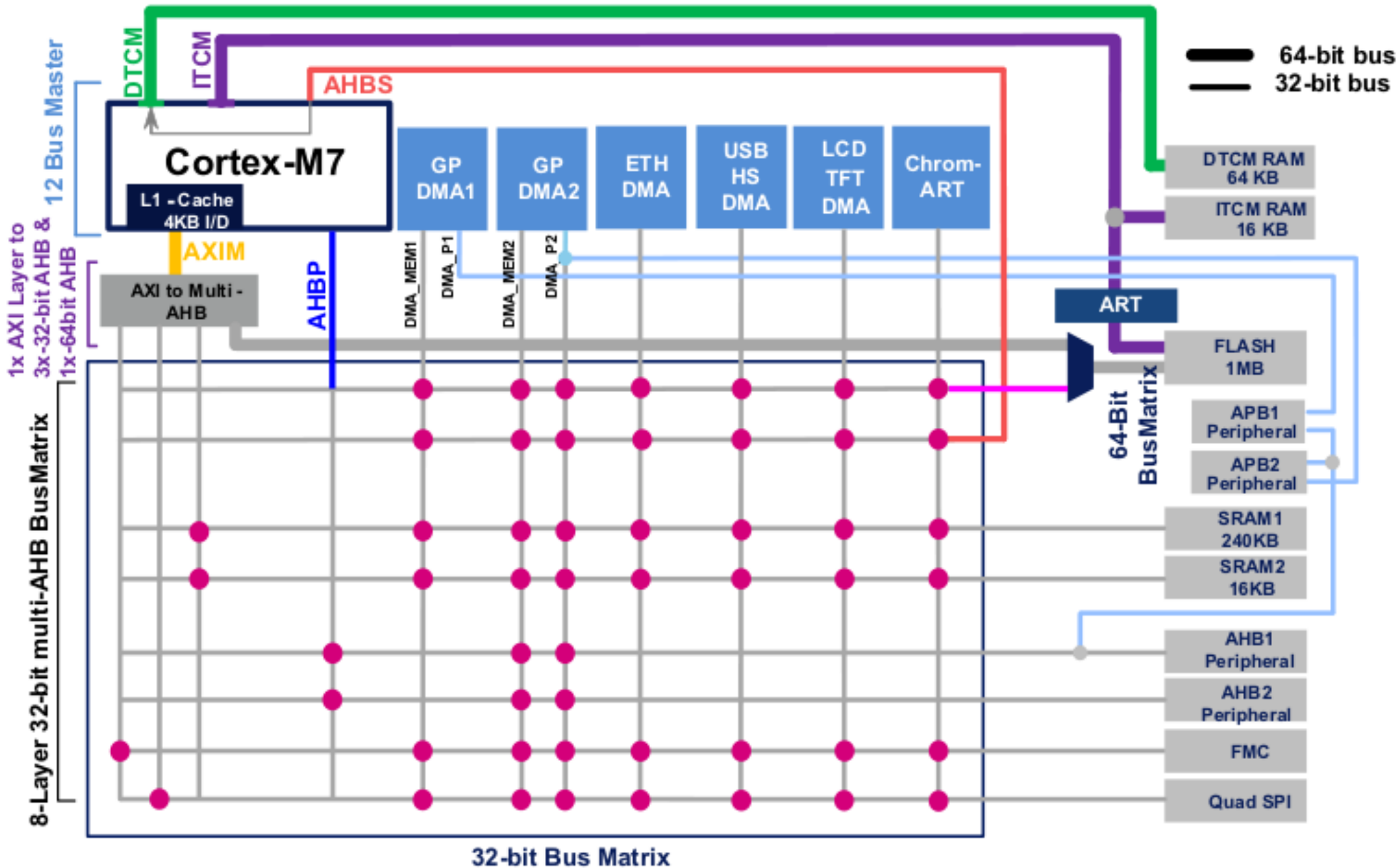
Register Set

Name		Functions (and banked registers)	
R0		General-purpose register	Low registers
R1		General-purpose register	
R2		General-purpose register	
R3		General-purpose register	
R4		General-purpose register	
R5		General-purpose register	
R6		General-purpose register	
R7		General-purpose register	High registers
R8		General-purpose register	
R9		General-purpose register	
R10		General-purpose register	
R11		General-purpose register	
R12		General-purpose register	
R13 (MSP)	R13 (PSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)	
R14		Link Register (LR)	
R15		Program Counter (PC)	

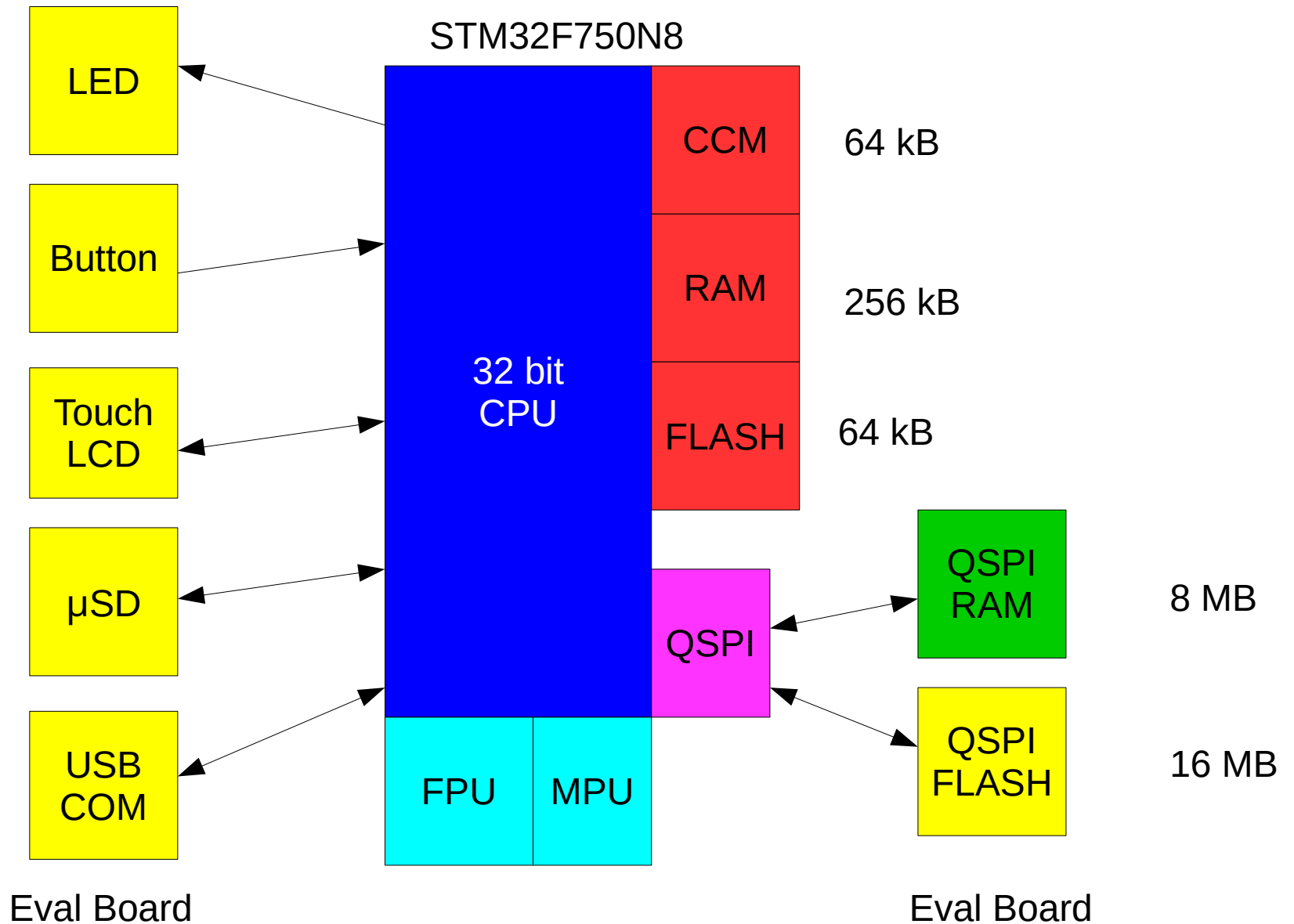
Memory Architecture



STM32F7 CM7 Bus Systems

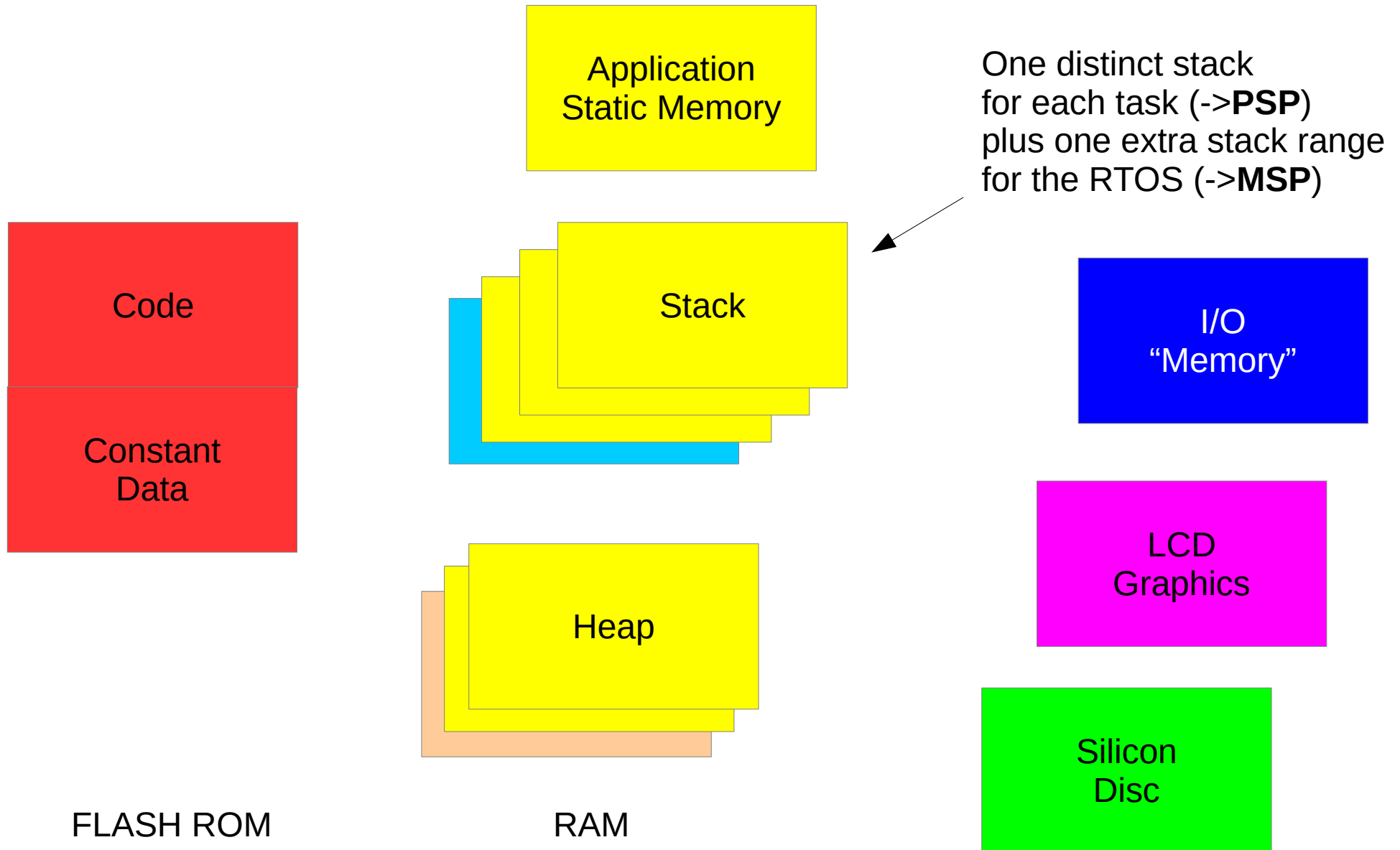


STM32F5708-DK

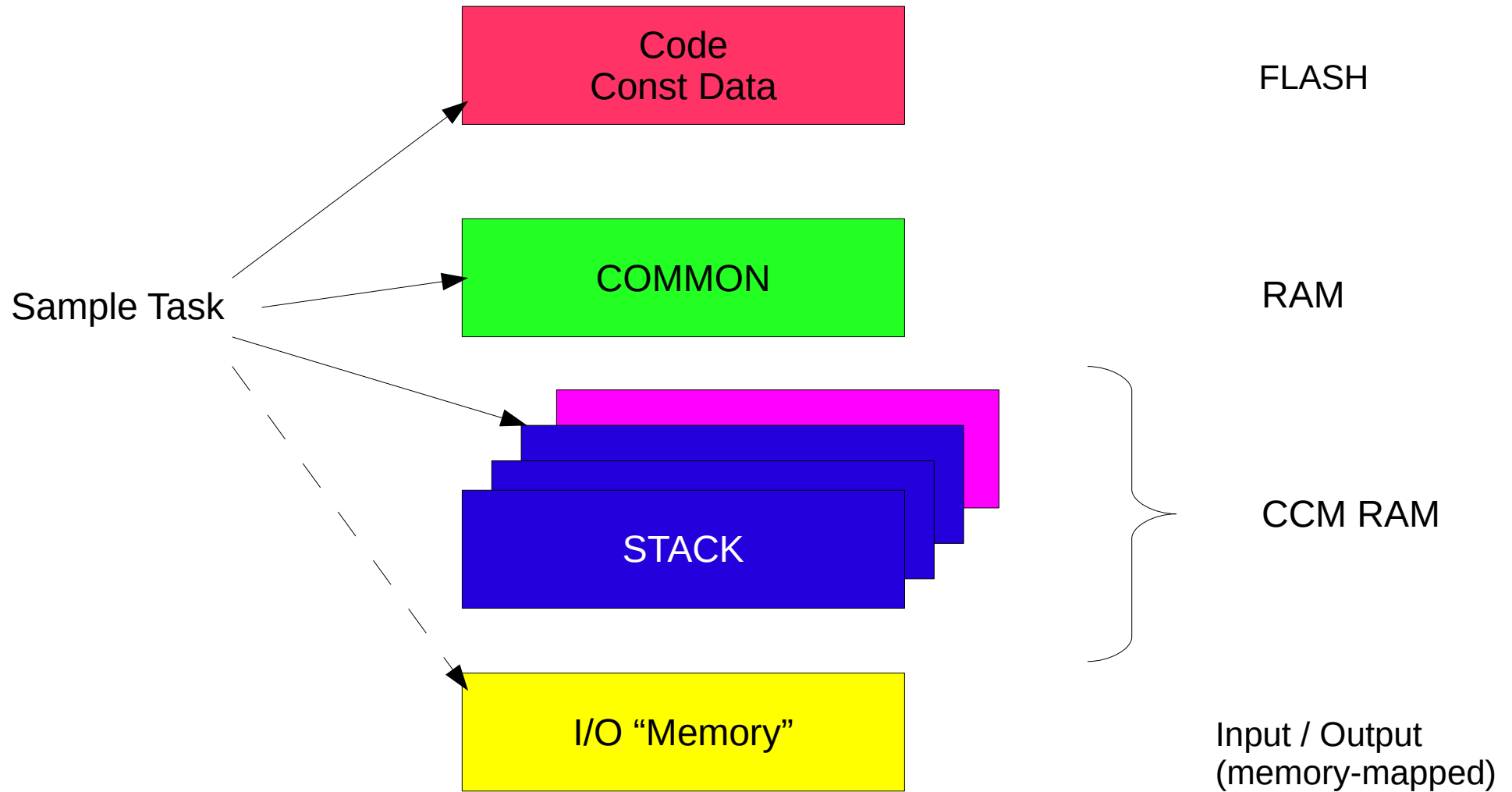


Memory “Types”

Application View



Memory Layout



Example:

FreeRTOS MPU support

Fixed Memory Regions:

- | | |
|---------------------|----------------------------|
| 1) User Code + Libs | (xr unprivileged) |
| 2) System Code | (xr privileged) |
| 3) System Data | (r/w privileged) |
| 4) I/O memory | (r/w unprivileged) |
| 5) Task Stack | (r/w x-never unprivileged) |

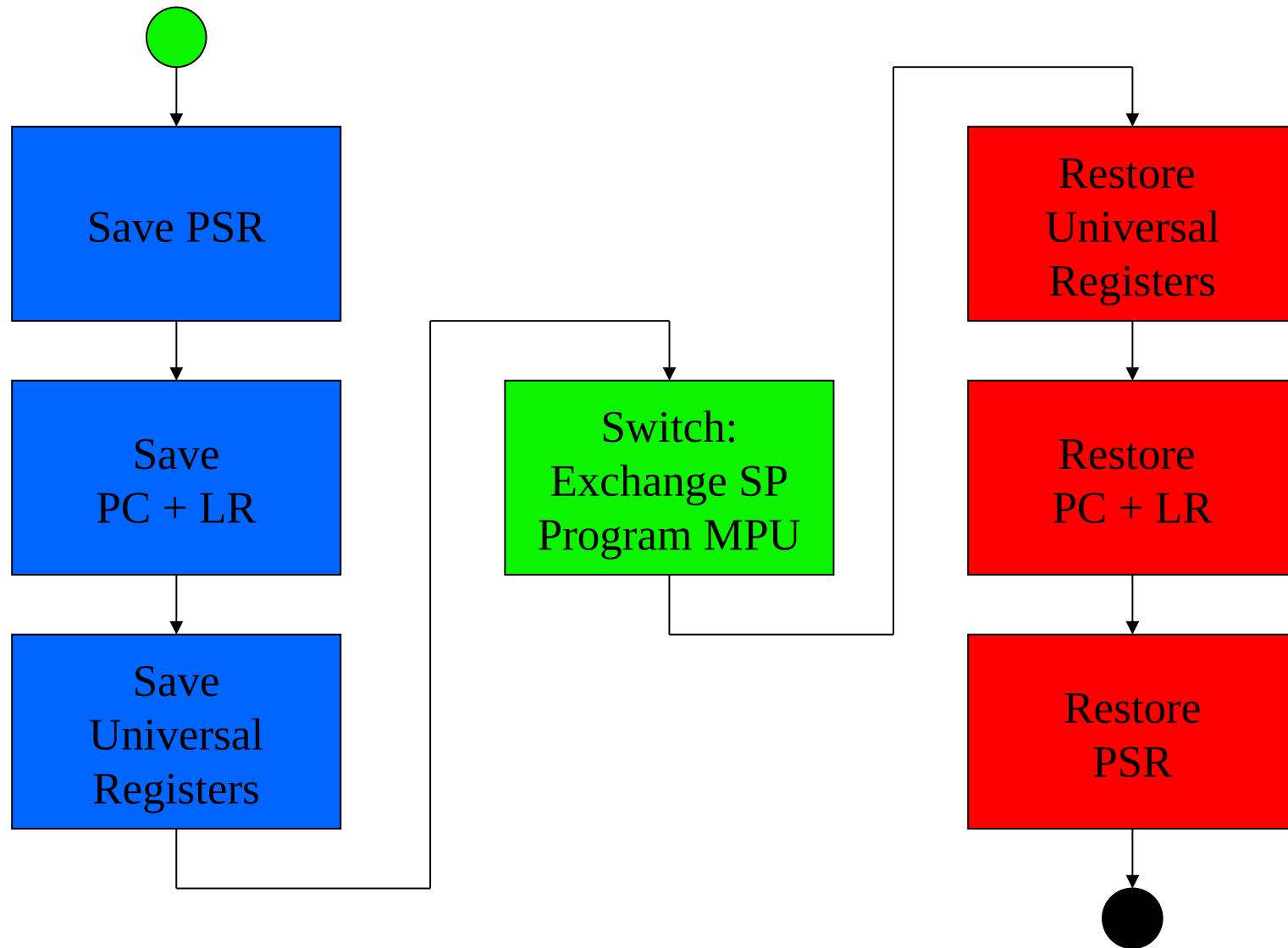
Distinct for each task

6-8) User-Configurable Memory Regions

Example “COMMON”

Three more regions for configurable access rights,
distinct for every task.

Thread Switch / Context Switch



Cortex M Processors can do almost all of that automatically by using their SVC / return from interrupt mechanisms.

Preemptive context switch

```
/** @brief This function handles External line0 interrupt request. */
void EXTI15_10_IRQHandler(void)
{
    portBASE_TYPE TaskWoken = pdFALSE;

    /* reset EXTI I/O interrupt latch */
    EXTI_ClearITPendingBit(KEY_BUTTON_EXTI_LINE);

    /* disable further interrupts for a while */
    NVIC_InitStructure.NVIC_IRQChannelCmd = DISABLE;
    NVIC_Init(&NVIC_InitStructure);

    /* wake up task */
    xQueueSendFromISR( EXTIqueue, 0, &TaskWoken);

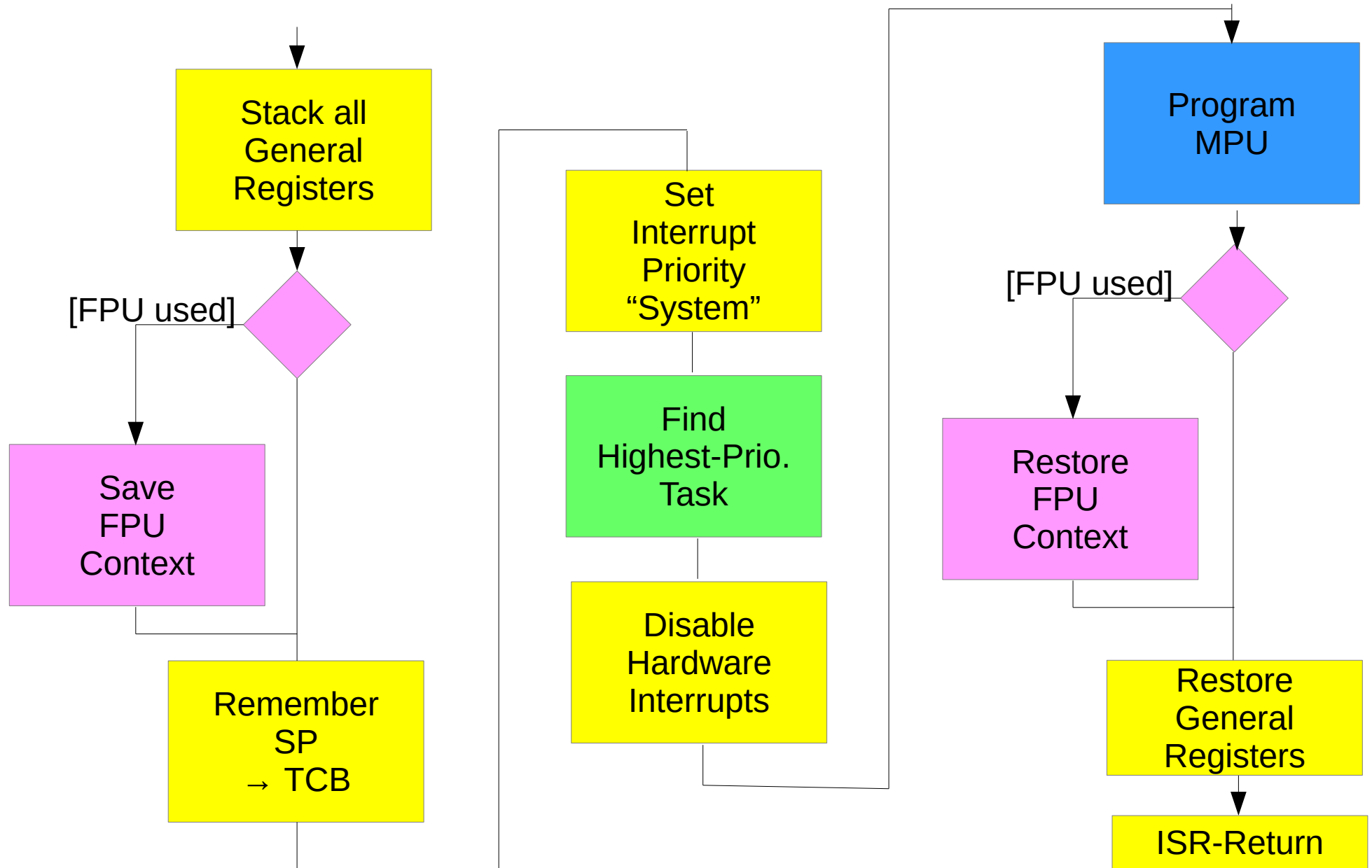
    /* trigger timer callback */
    xTimerStartFromISR( de_bounce_Timer, &TaskWoken);

    /* preempt the active task if necessary */
    portEND_SWITCHING_ISR( TaskWoken);
}
```



```
#define portEND_SWITCHING_ISR( xSwitchRequired ) \
    if( xSwitchRequired ) *(portNVIC_INT_CTRL) = portNVIC_PENDSVSET
```

Task Switch: PendSV-Handler

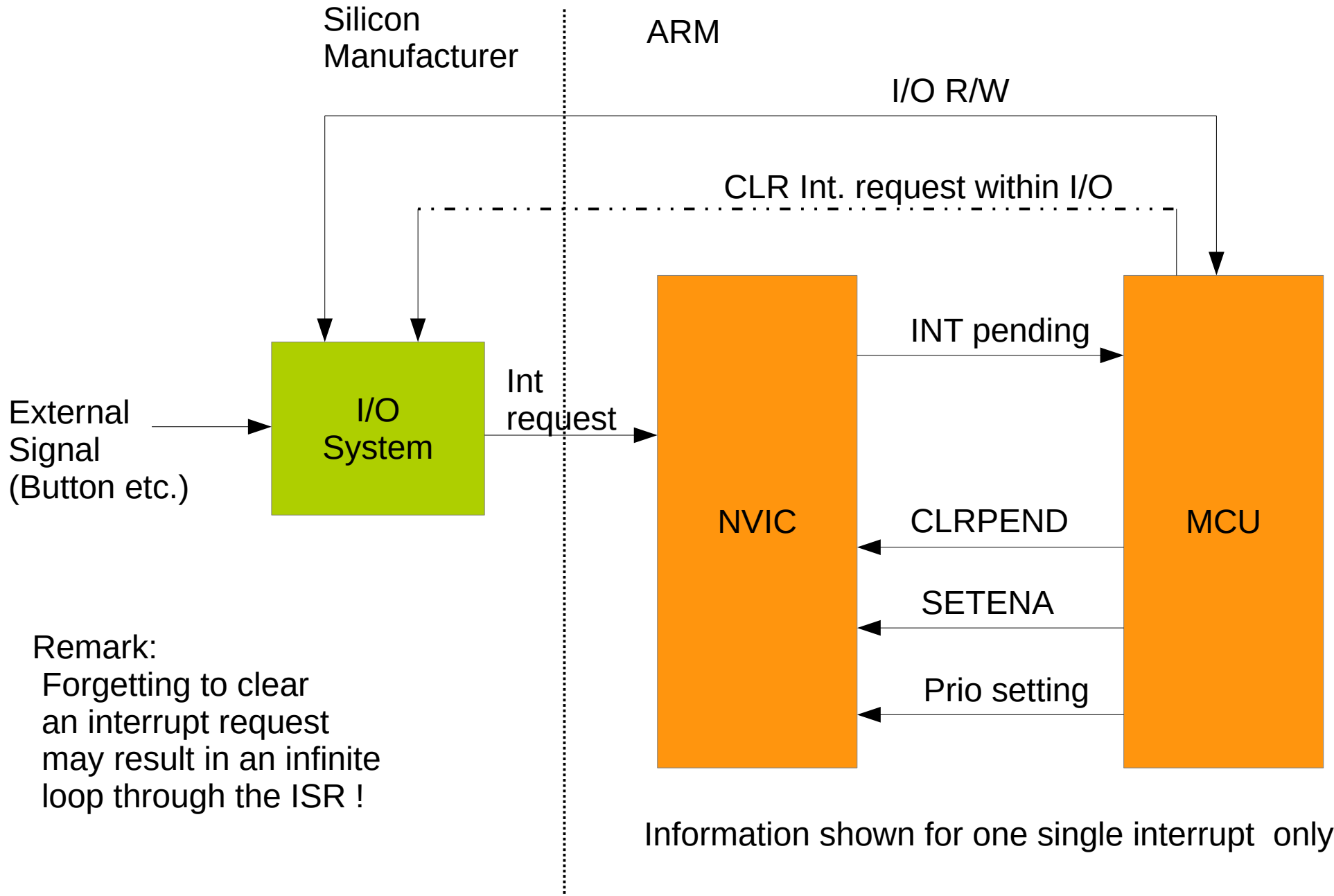


```

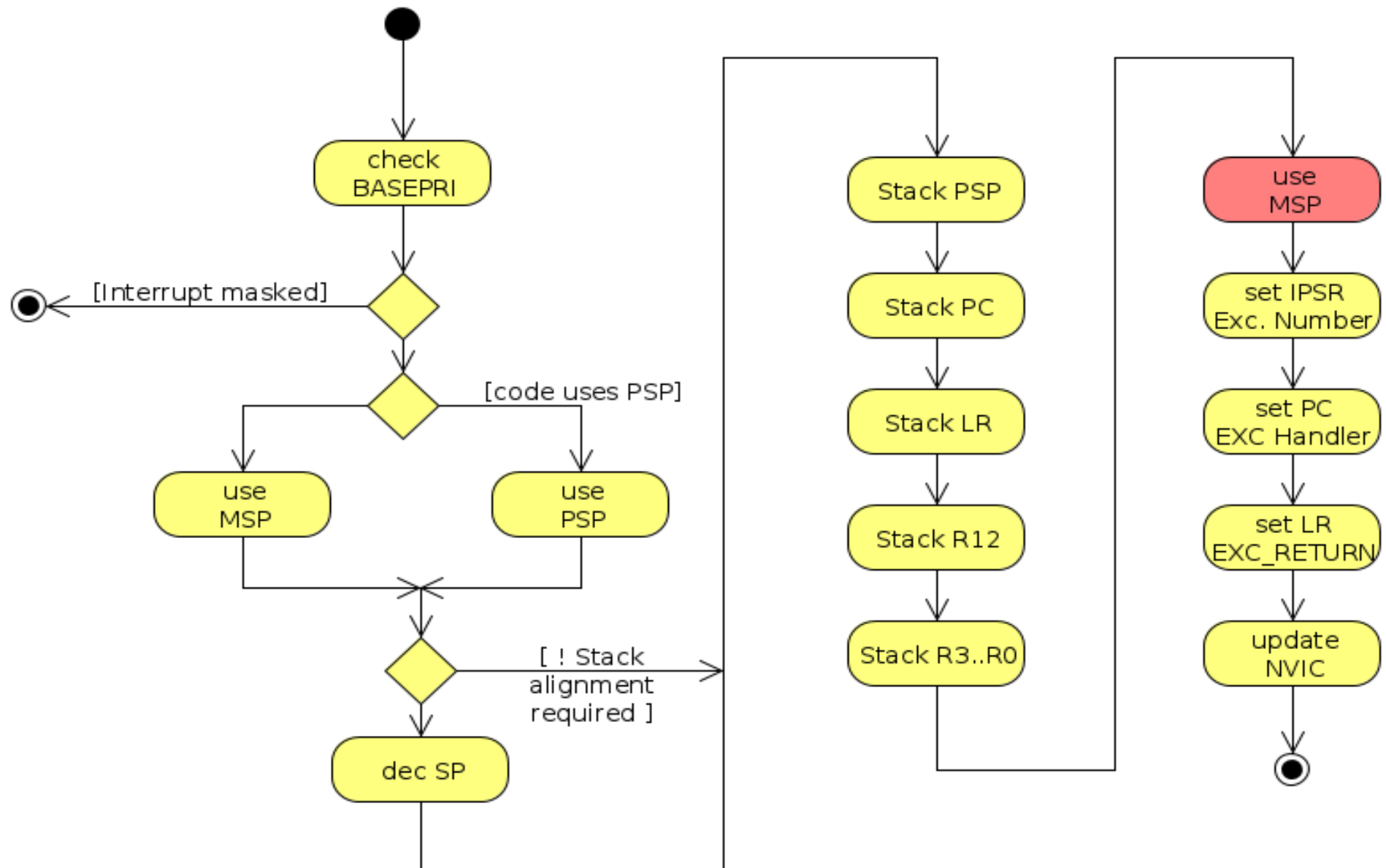
void xPortPendSVHandler( void ) /* This is a naked function. */
{
    __asm volatile
    (
        "      mrs r0, psp\n"
        "\n"
        "      ldr r3, pxCurrentTCBConst\n"
        "      ldr r2, [r3]\n"
        "\n"
        "      mrs r1, control\n"
        "      stmdb r0!, {r1, r4-r11}\n"
        "      ldr r1, [r2, #40]\n"
        "      tst r1, #0x01\n"
        "      ittt ne\n"
        "      vstmbdne r0!, {s0-s31}\n"
        "      vmrsne r1, fpscr\n"
        "      stmbdne r0!, {r1}\n"
        "\n"
        "      str r0, [r2]\n"
        "\n"
        "      stmdb sp!, {r3, r14}\n"
        "      mov r0, %0\n"
        "      msr basepri, r0\n"
        "      bl vTaskSwitchContext\n"
        "      mov r0, #0\n"
        "      msr basepri, r0\n"
        "      ldmbia sp!, {r3, r14}\n"
        "\n"
        "\n"
        "      ldr r1, [r3]\n"
        "      ldr r0, [r1]\n"
        "      add r1, r1, #4\n"
        "      ldr r2, =0xe000ed9c\n"
        "      ldmbia r1!, {r4-r11}\n"
        "      stmia r2!, {r4-r11}\n"
        "\n"
        "      ldr r3, [r1, #4]\n"
        "      tst r3, #0x01\n"
        "      ittt ne\n"
        "      ldmiane r0!, {r1}\n"
        "      vmrsne fpscr, r1\n"
        "      vldmiane r0!, {s0-s31}\n"
        "      ldmbia r0!, {r3, r4-r11}\n"
        "\n"
        "\n"
        "      msr control, r3\n"
        "\n"
        "      msr psp, r0\n"
        "      bx r14\n"
        "\n"
        "\n"
        ".align 2\n"
        "pxCurrentTCBConst: .word pxCurrentTCB\n"
        "::"i"(configMAX_SYSCALL_INTERRUPT_PRIORITY)
    );
}

```

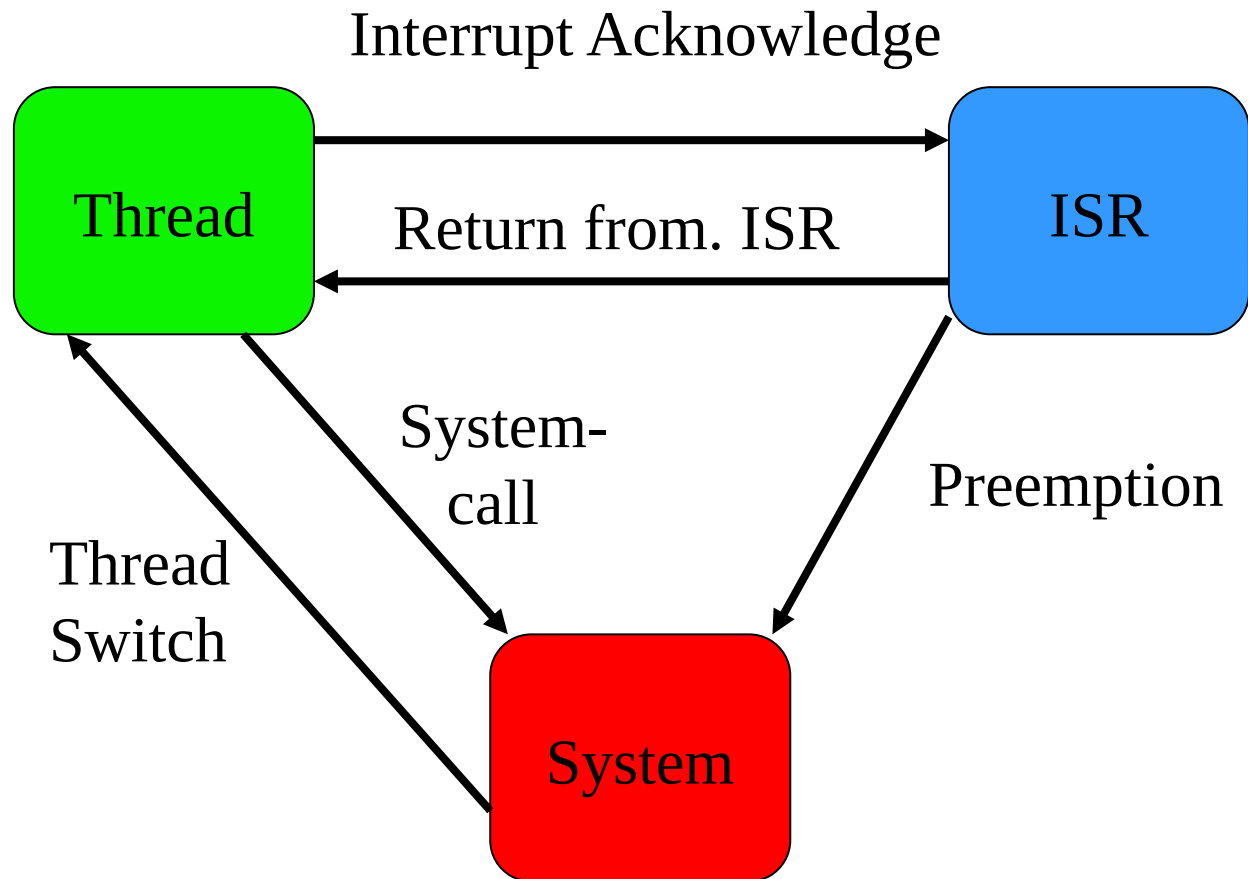
Interrupt Hardware



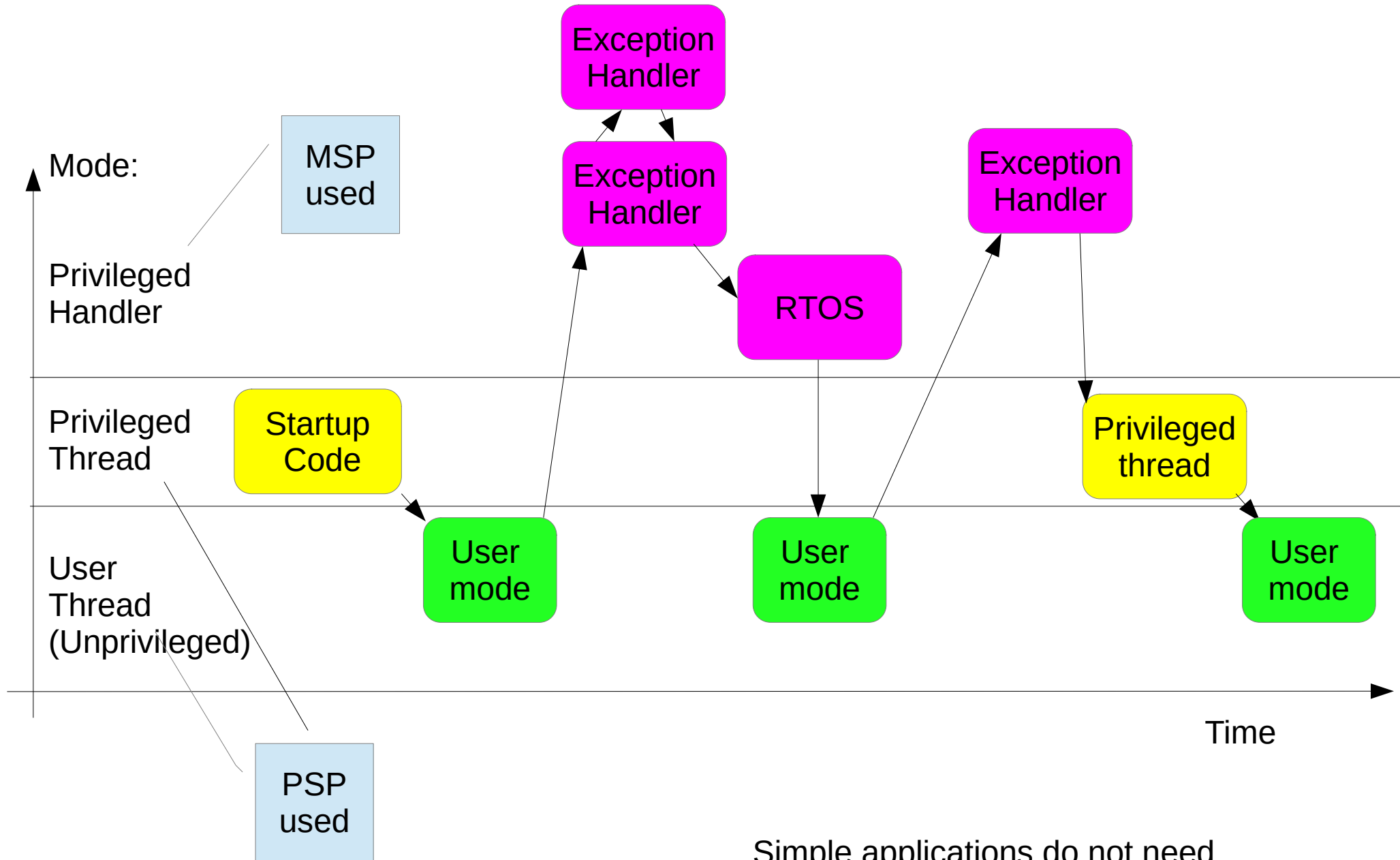
Interrupt / Exception Sequence

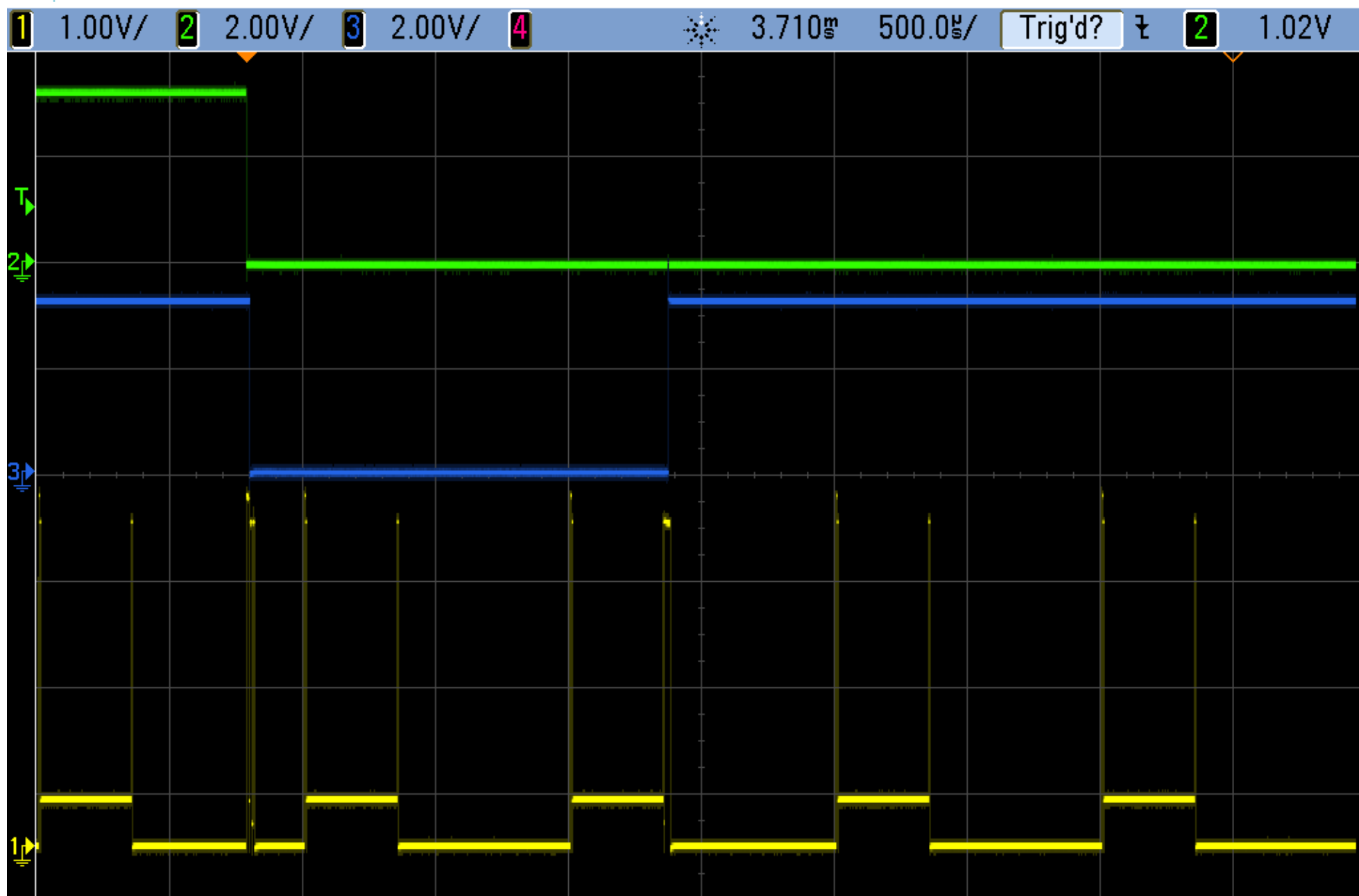


System State



CM3 Operation Mode





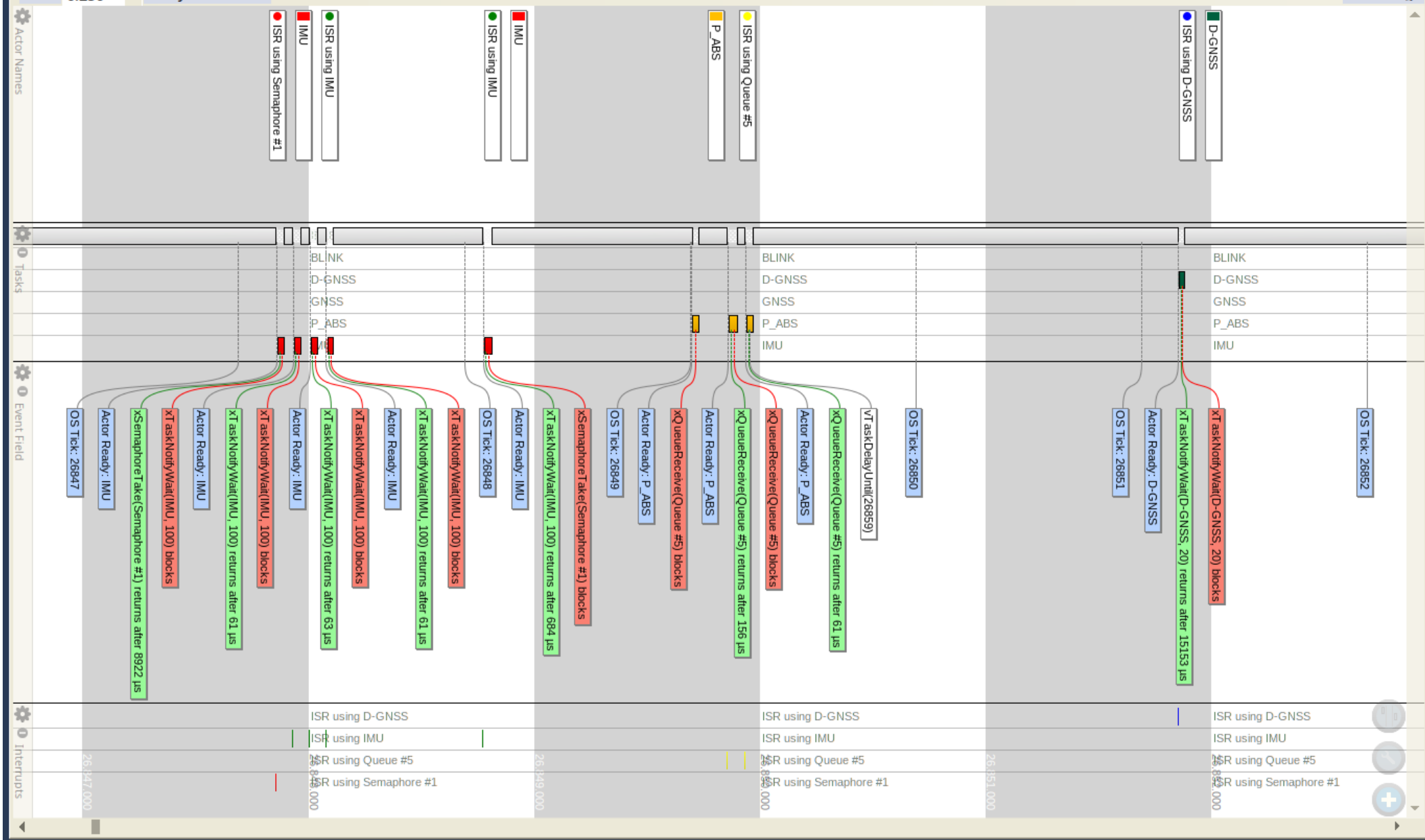
Edge Trigger Menu

Source
2

Slope
↑

 6,250 Sync View

ing



Float number ranges

IEEE single precision (32 bit)

float

$\pm 10^{-37} \dots \pm 10^{+38}$

Mantissa: 23 bit = approx. 7 significant decimal digits

IEEE double precision (64 bit)

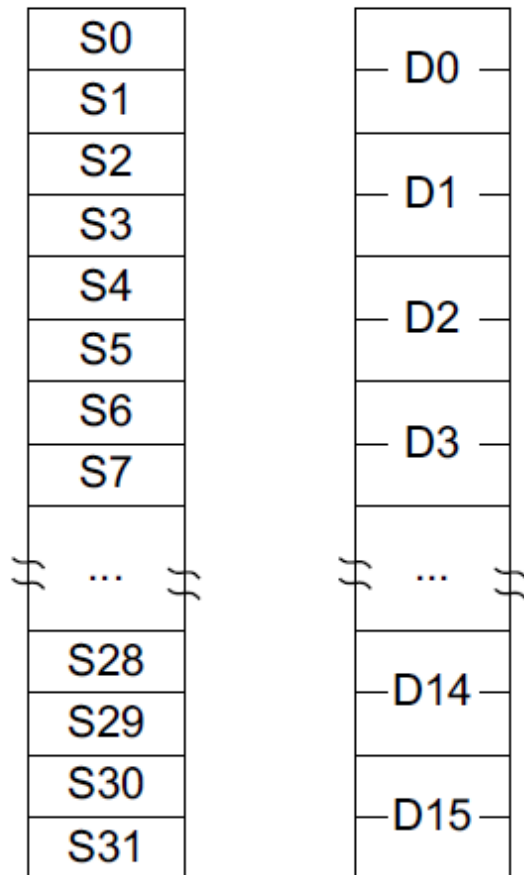
double

$\pm 10^{-307} \dots \pm 10^{+308}$

Mantissa: 52 bit = 15.65 significant decimal digits

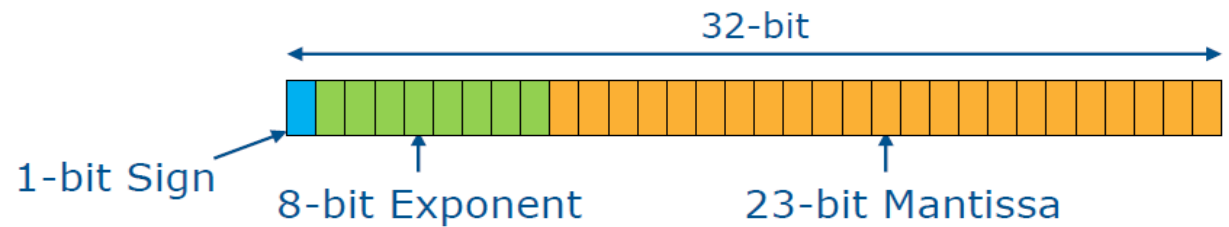
(Distance Earth-Moon: 384401 km / $2^{52} = 0.085$ nm !)

Floating Point Support

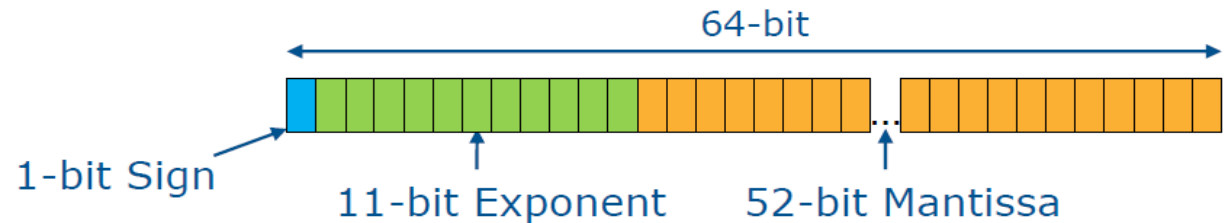


FPU register bank

Single precision : 32-bit coding



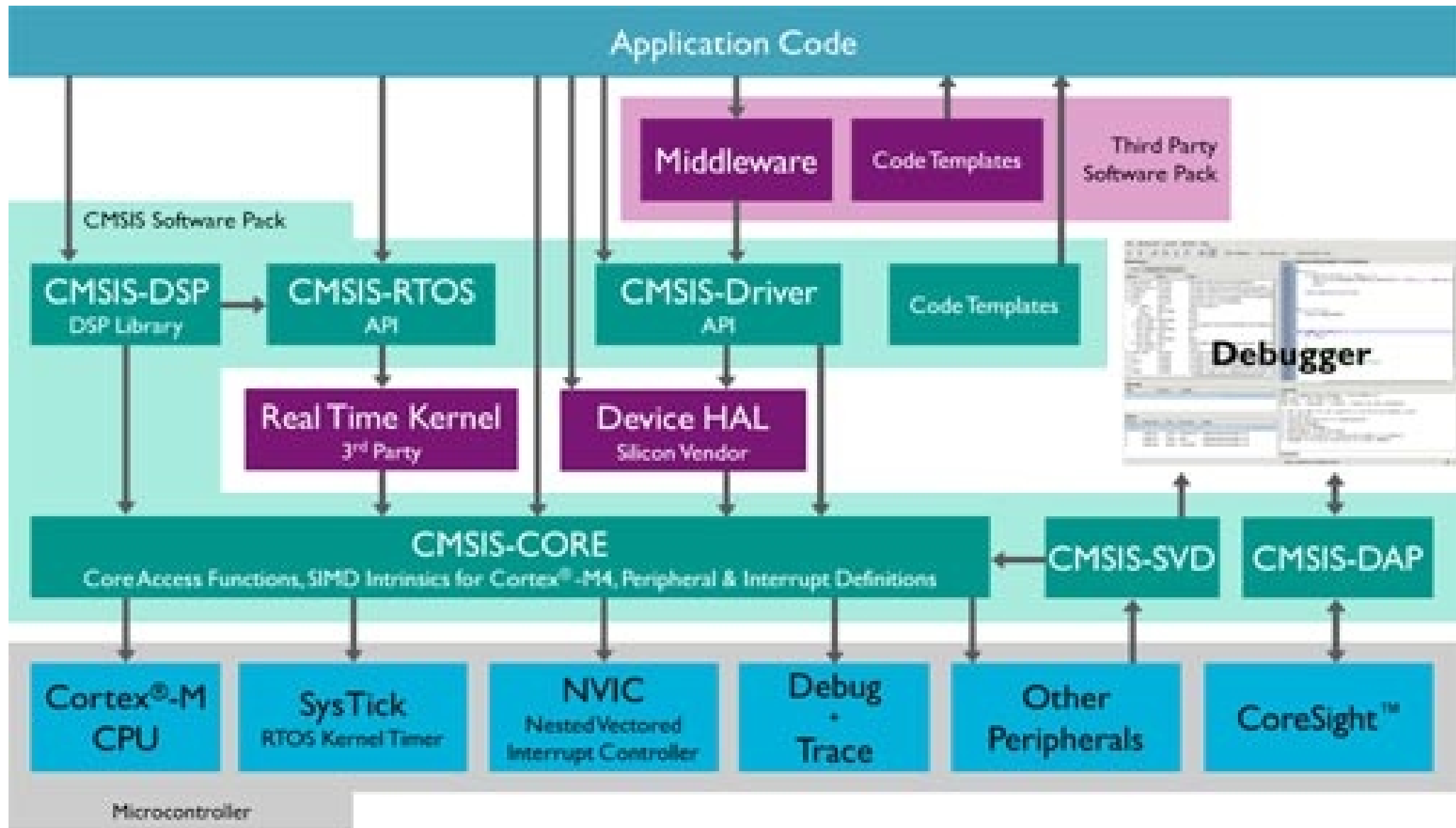
Double precision : 64-bit coding



Cortex M4F adds 32 16-bit float registers to the integer register bank.

CMSIS

Cortex Microcontroller Software Interface Standard



Communication / Synchronization



Mutex



Mutual Exclusion



Queue



Communication



Counting Semaphore



Synchronisation



Binary Semaphore



Synchronisation



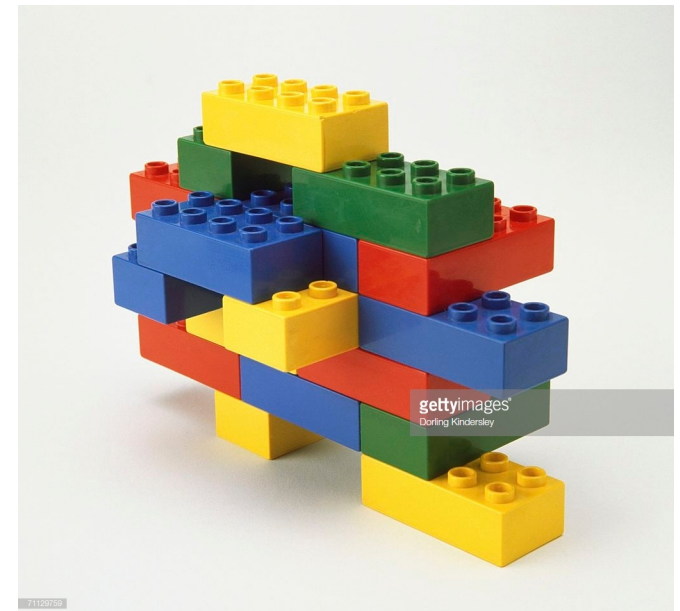
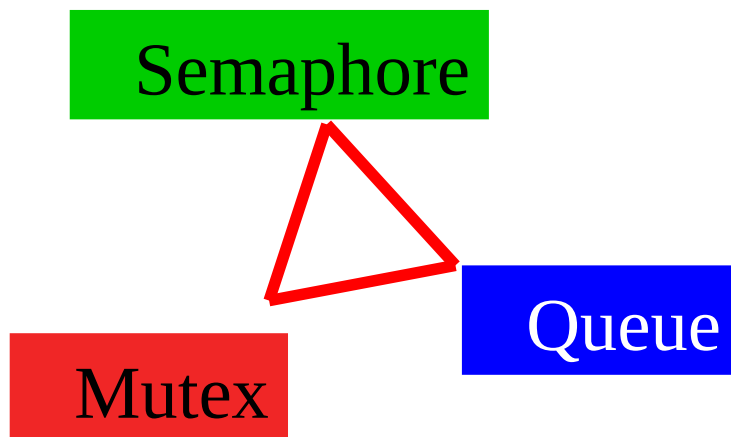
Timer



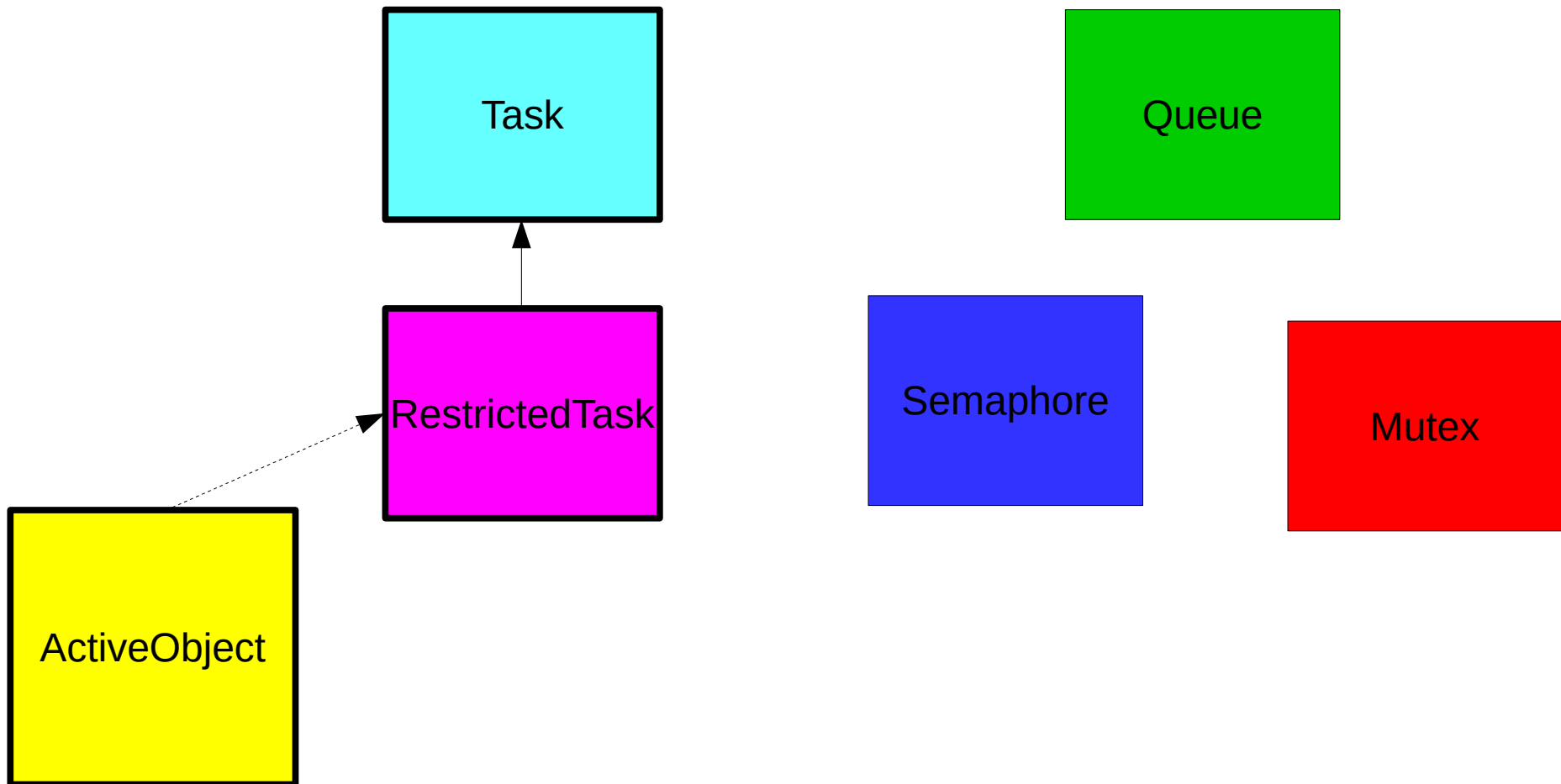
Synchronisation

Using FreeRTOS

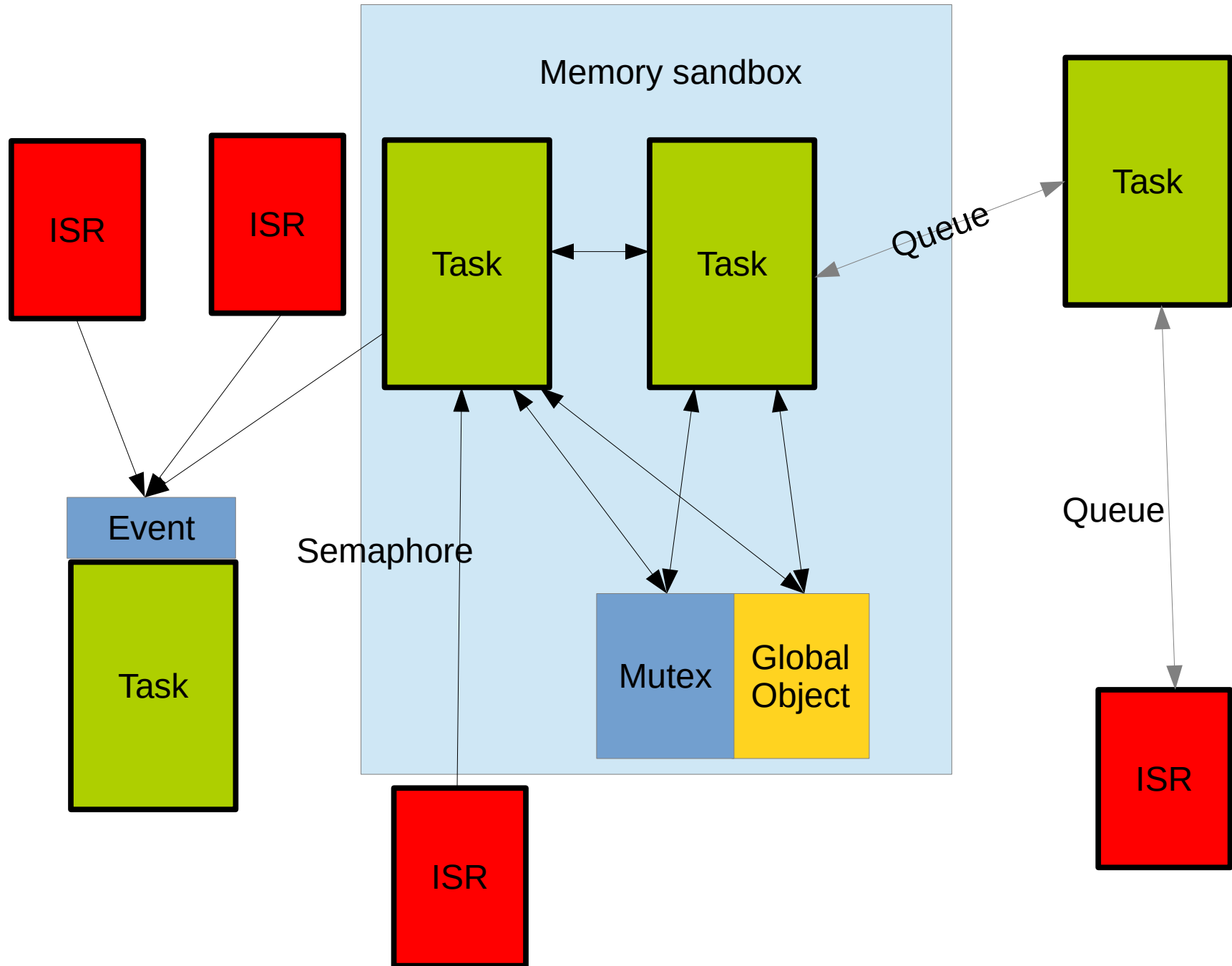
- Delay / Timer
- Queue
- Counting / Binary Semaphore
- Mutex



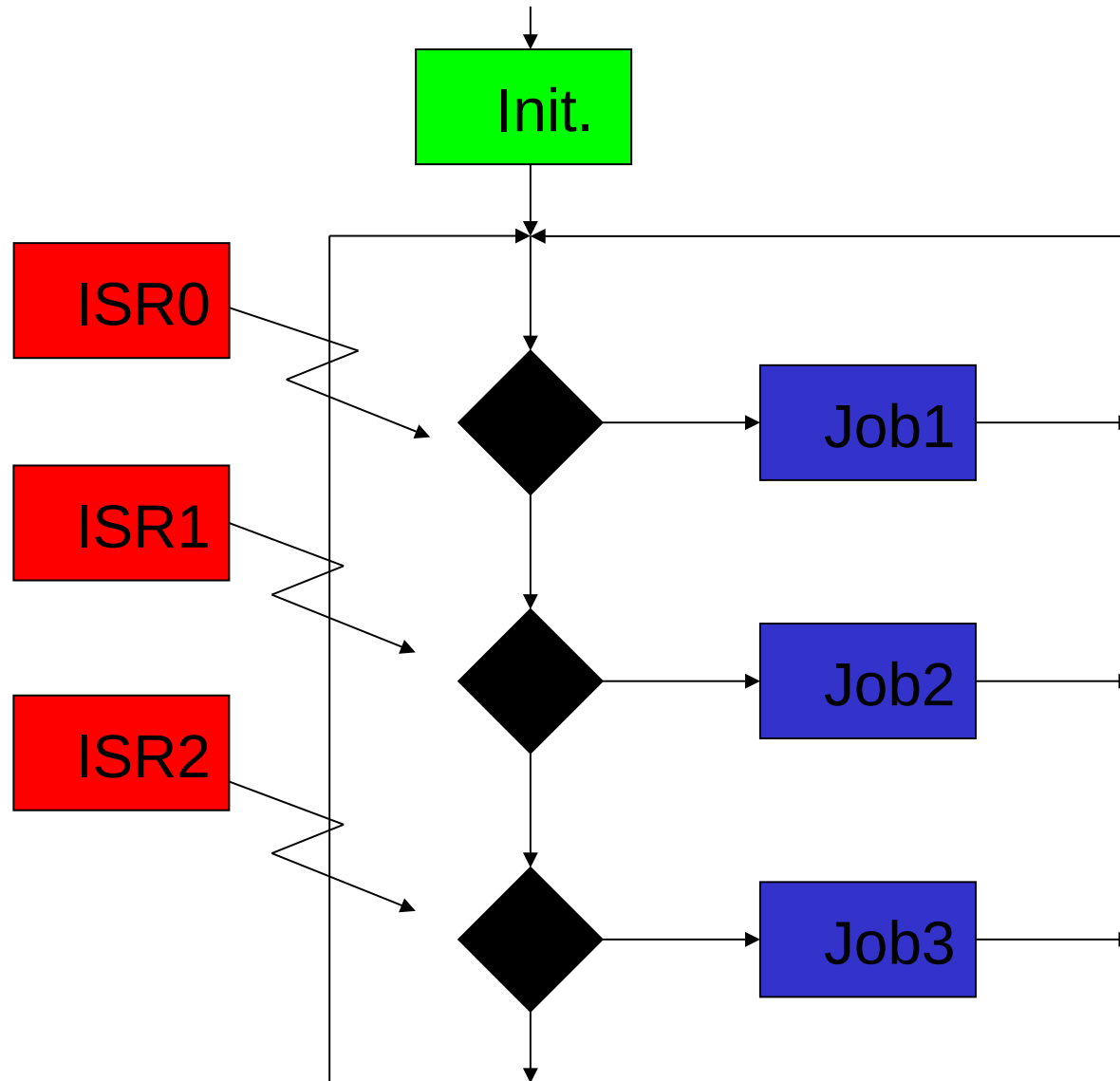
FreeRTOS Wrapper Object Oriented Runtime Environment



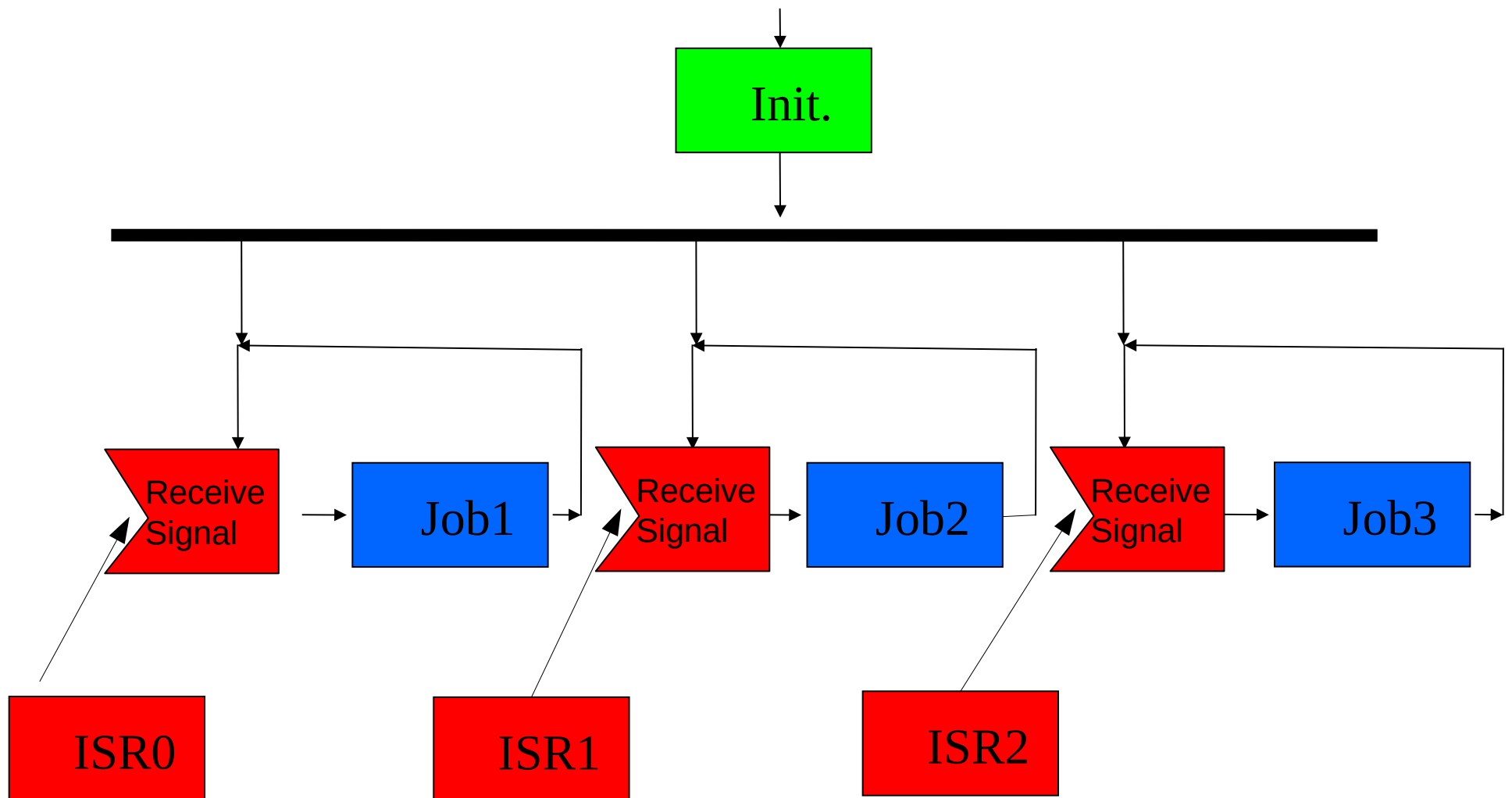
RTOS System: Object Model



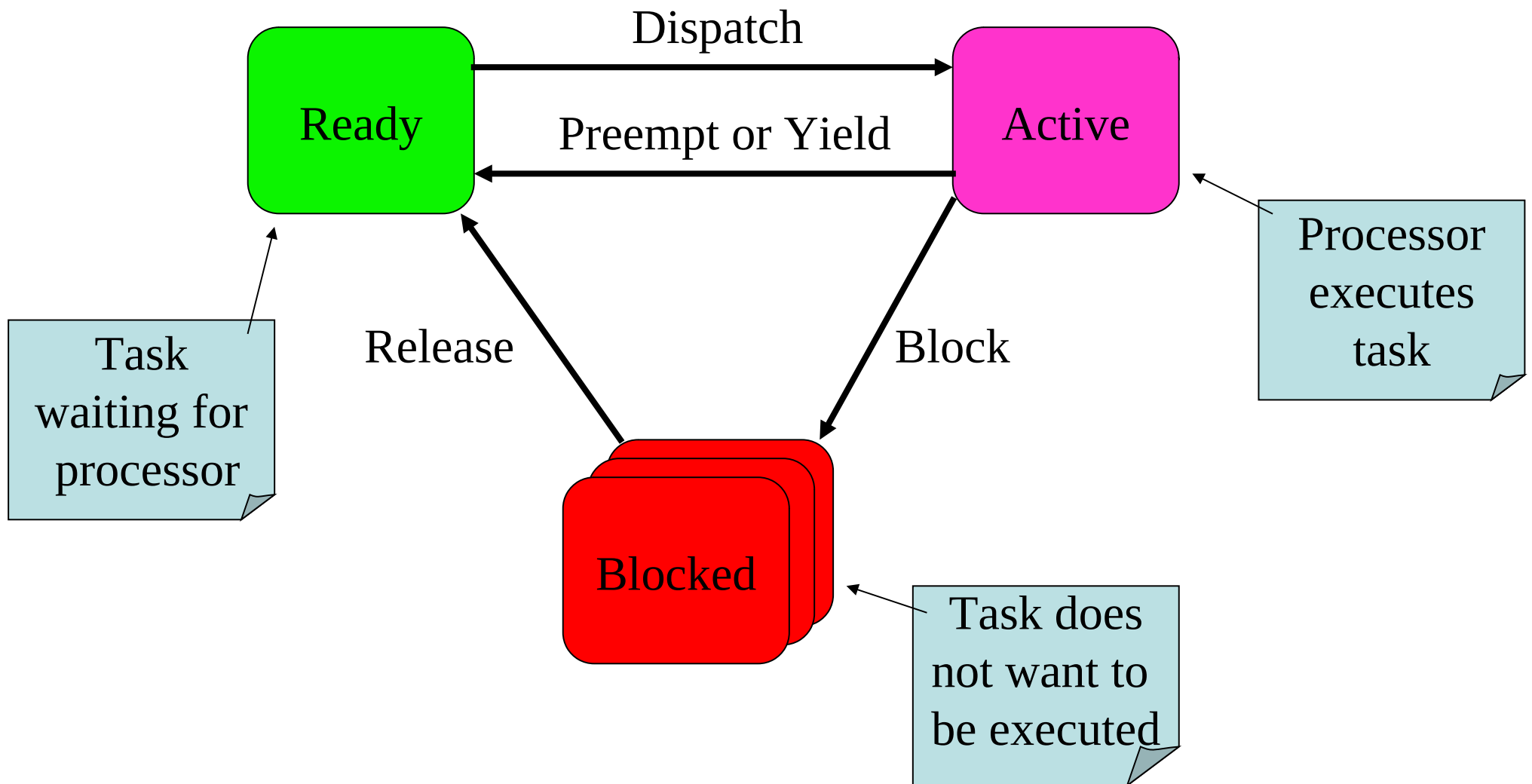
Interrupt-triggered Super Loop Architecture



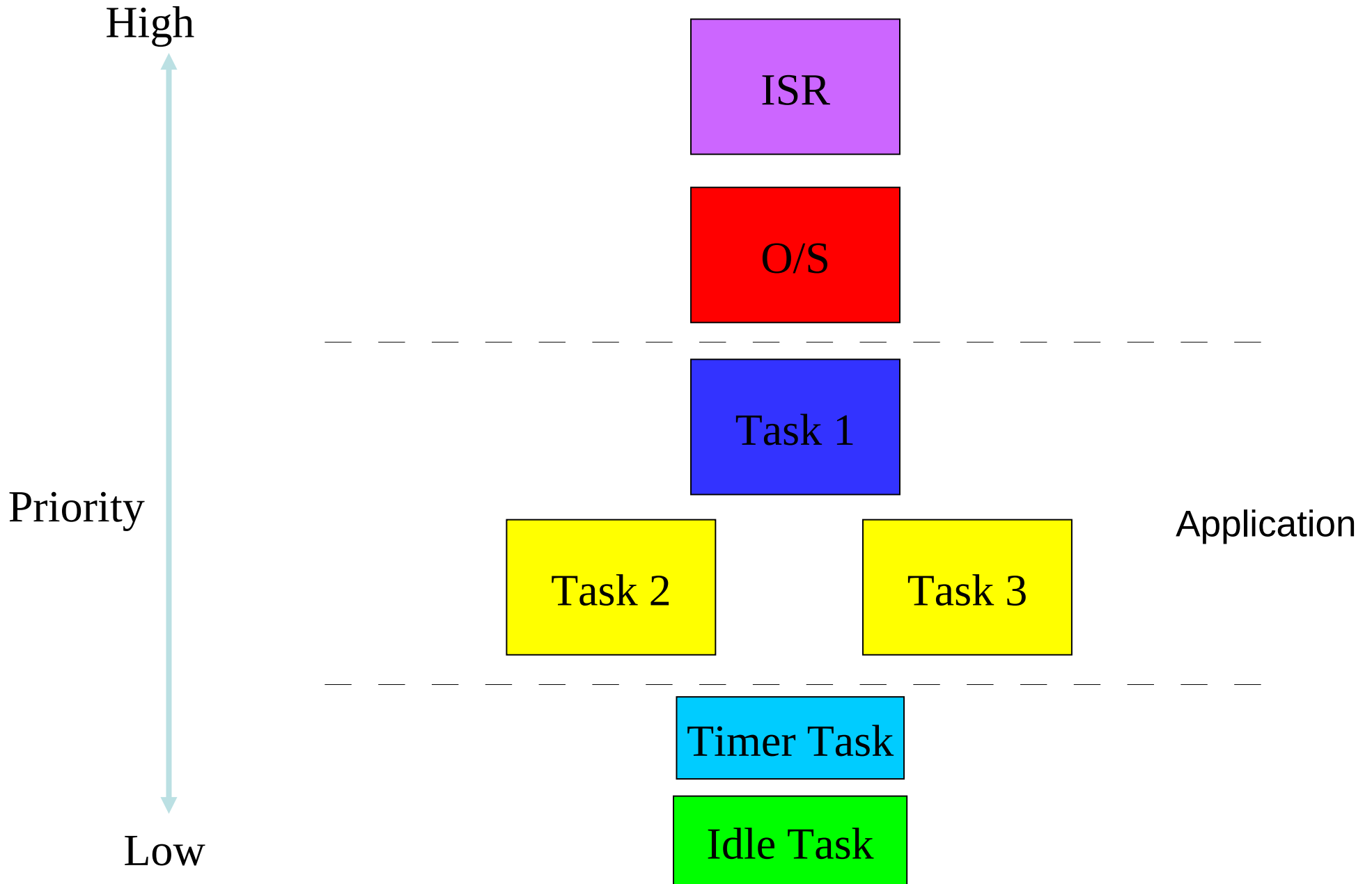
Interrupts + Multi-Threading



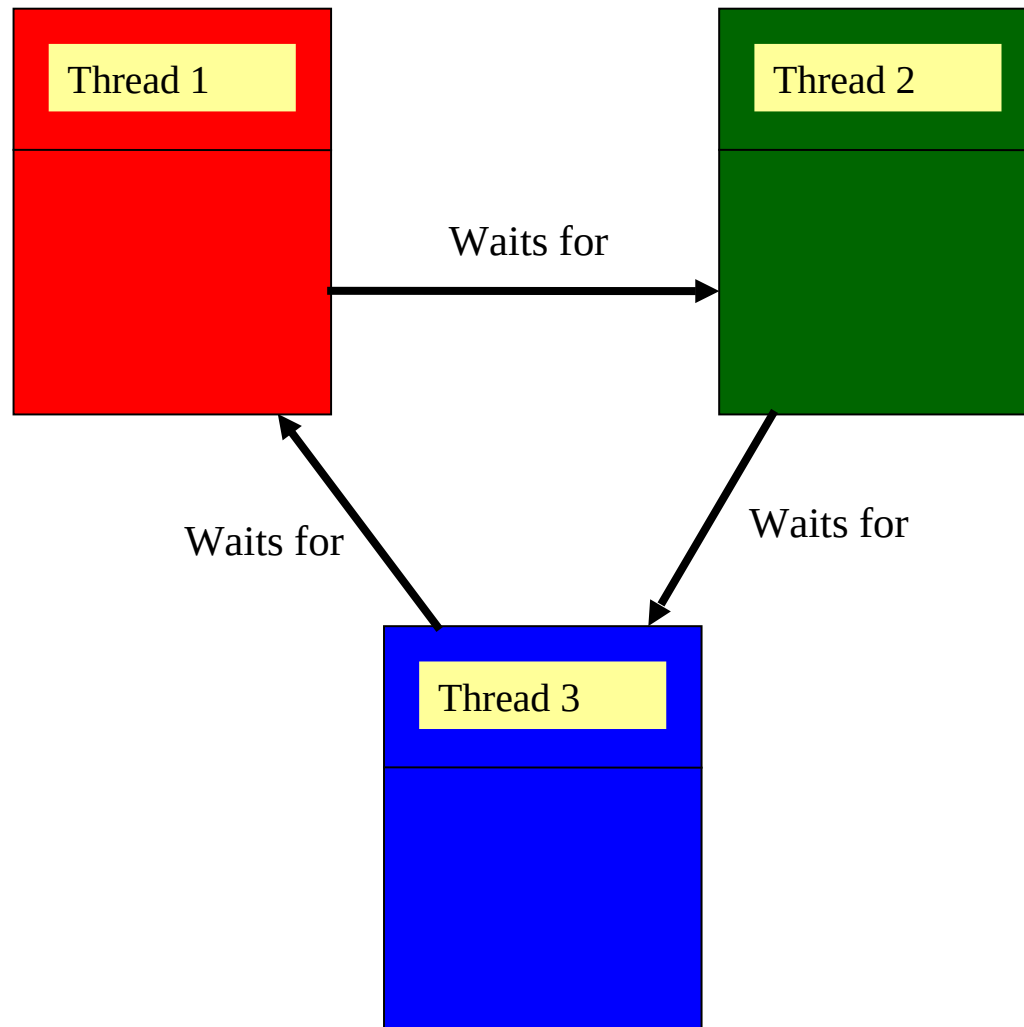
Task State



Priority



Deadlock



Priority Inversion

Thread 3
locks resource

Thread 1
preempts thread 3
trying to lock
resource, blocking

Thread 2 running,
preventing thread 3
to release resource

Thread 1



Thread 2



Thread 3



Problem:
Thread 3 blocks
Thread 1 !

“IPCP”

Immediate Priority Ceiling Protocol

For every resource the task with the highest priority that wants to use the resource any time is searched

This task's priority is the **ceiling priority**.

If an active task wants to access a resource
and if its dynamic priority is presently higher than the
priority of all presently locked resources
then the access is granted
and the priority is lifted up to the **ceiling priority** for the time of the access
immediately.

Remark: This leads to a unnecessary high temporary priority for this task

Original Ceiling Priority Protocol

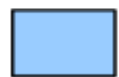
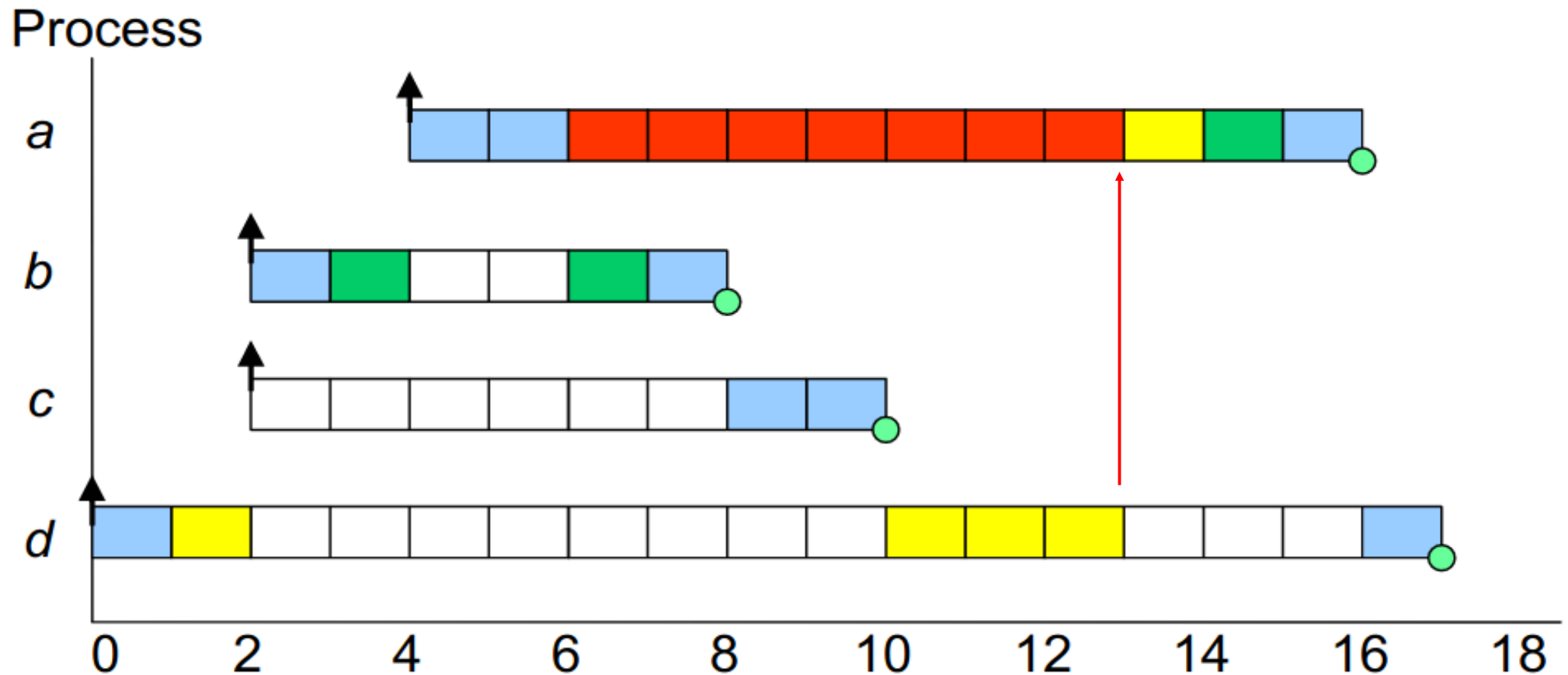
“OPCP”

For every resource the task with the highest priority that wants to use the resource is searched
This task's priority is the **ceiling priority**.

If a task wants to access a resource and if its priority is presently higher than the ceiling priority of all the currently locked resources
then the access will be granted and the scheduler records the ceiling priority of this resource.

If now another task wants to access the resource
then the priority of the first task will be lifted up to the ceiling priority for the time of the access.

Prioritized Resource Sharing



Executing



Preempted



Executing with Q locked



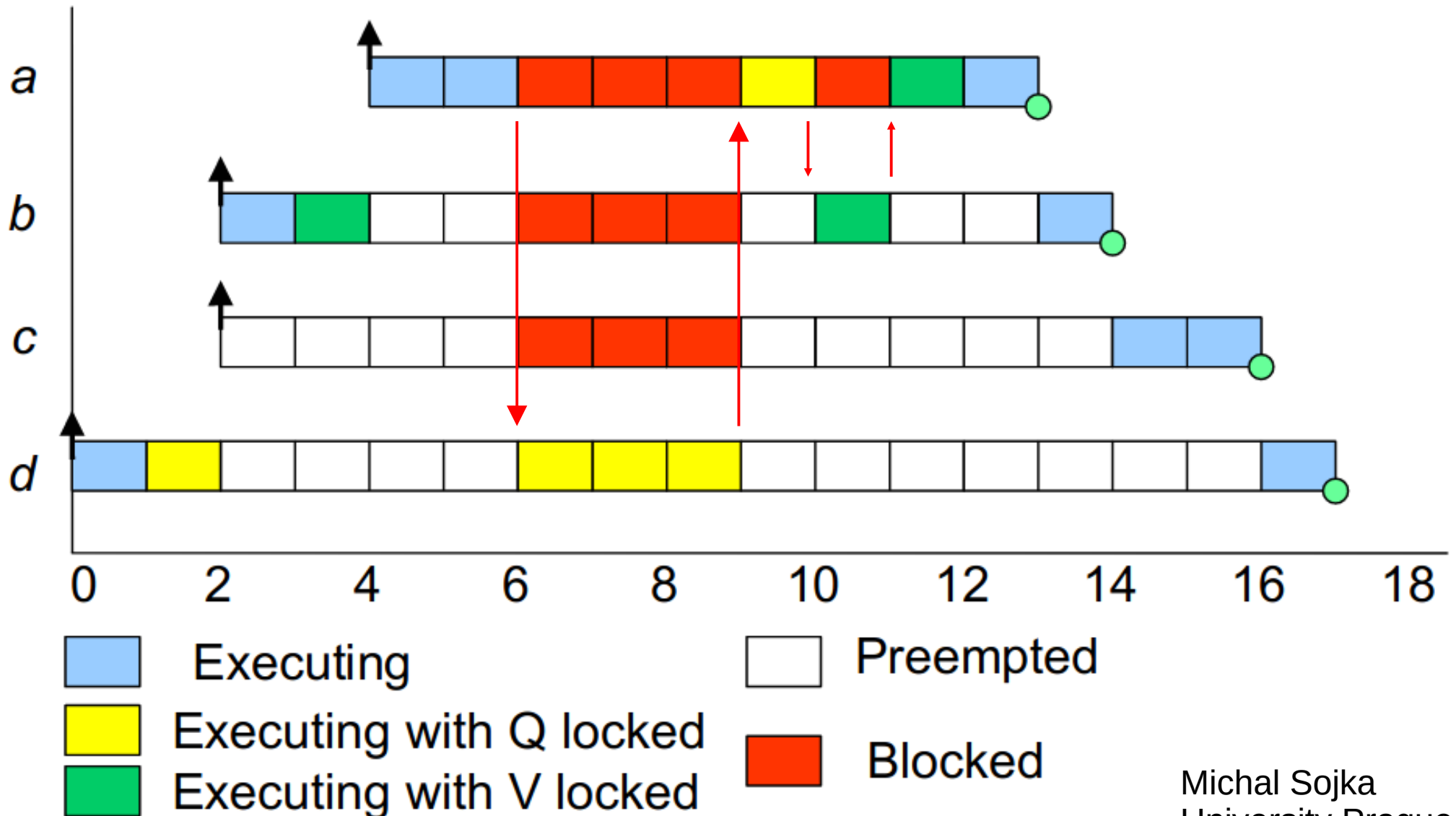
Blocked



Executing with V locked

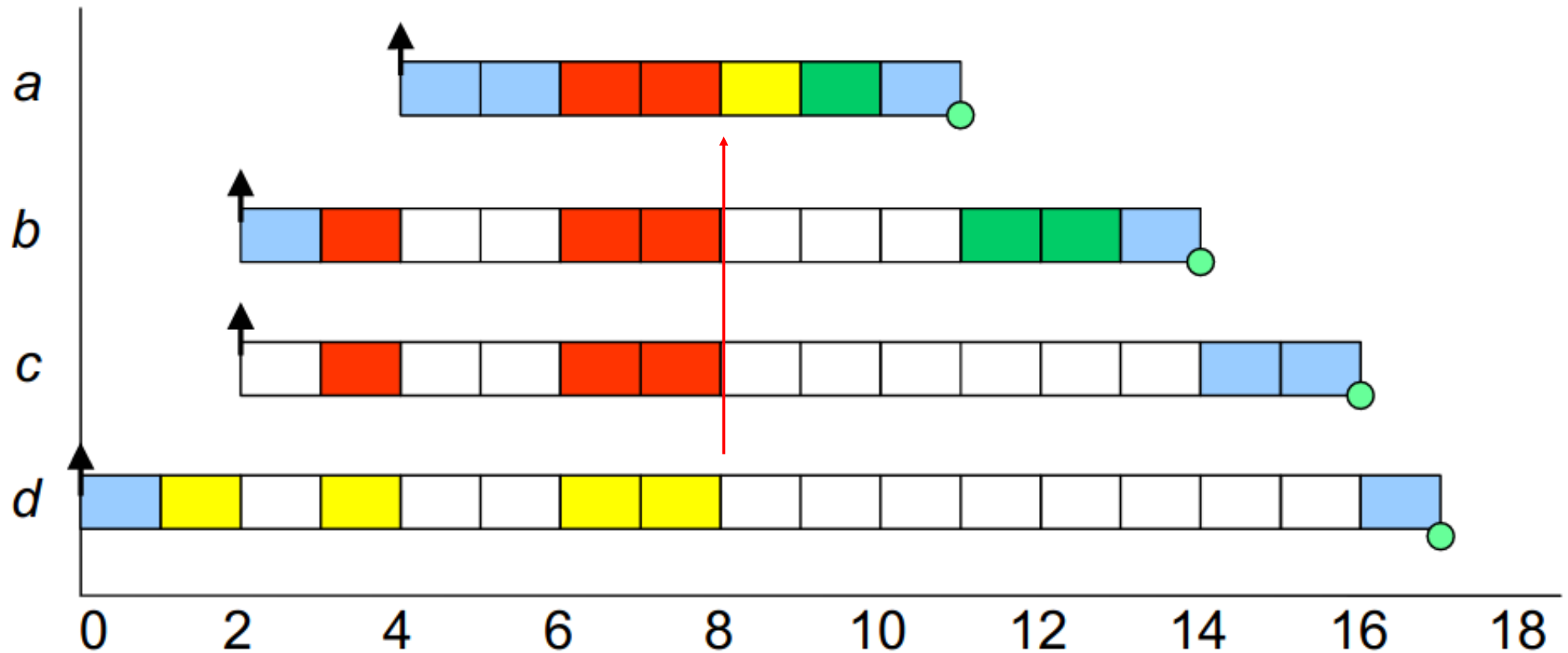
Priority Inheritance

Process



OPCP Inheritance

Process



Executing



Preempted



Executing with Q locked



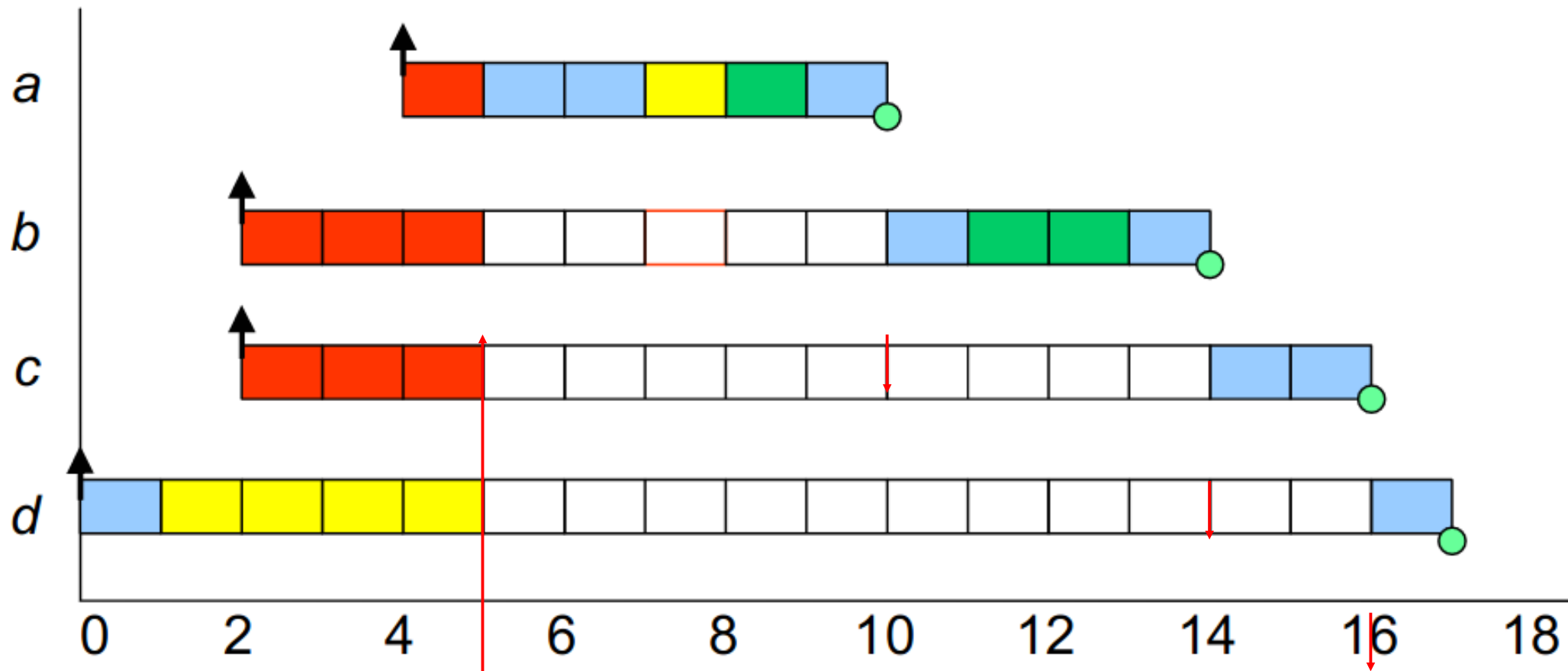
Blocked



Executing with V locked

IPCP Inheritance

Process



Executing



Preempted



Executing with Q locked



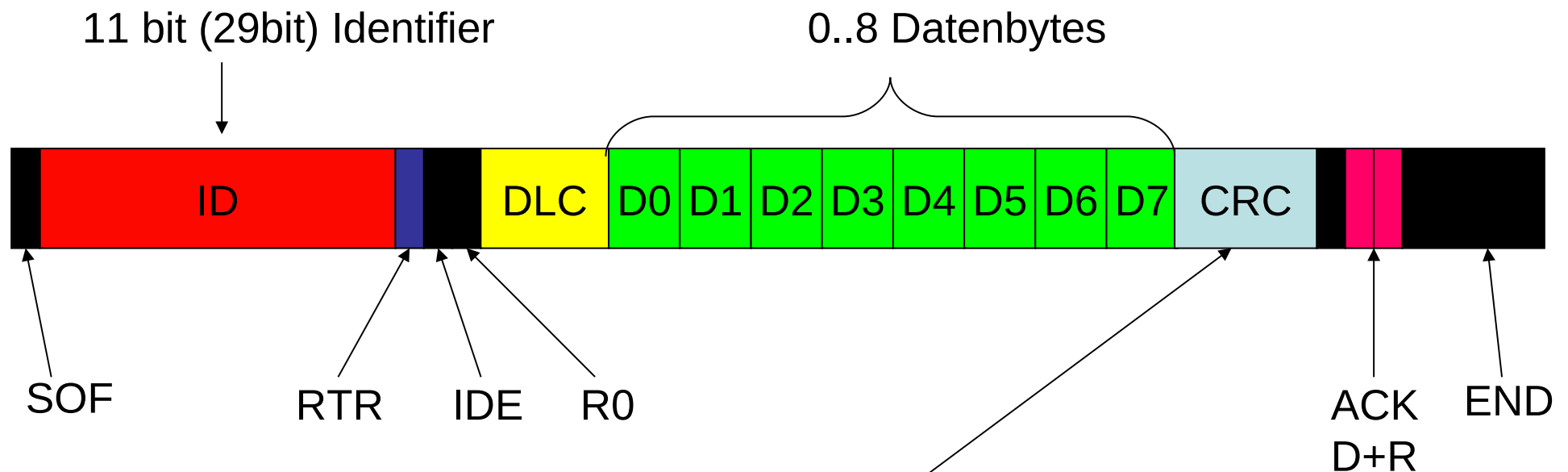
Blocked



Executing with V locked

CAN Telegramm

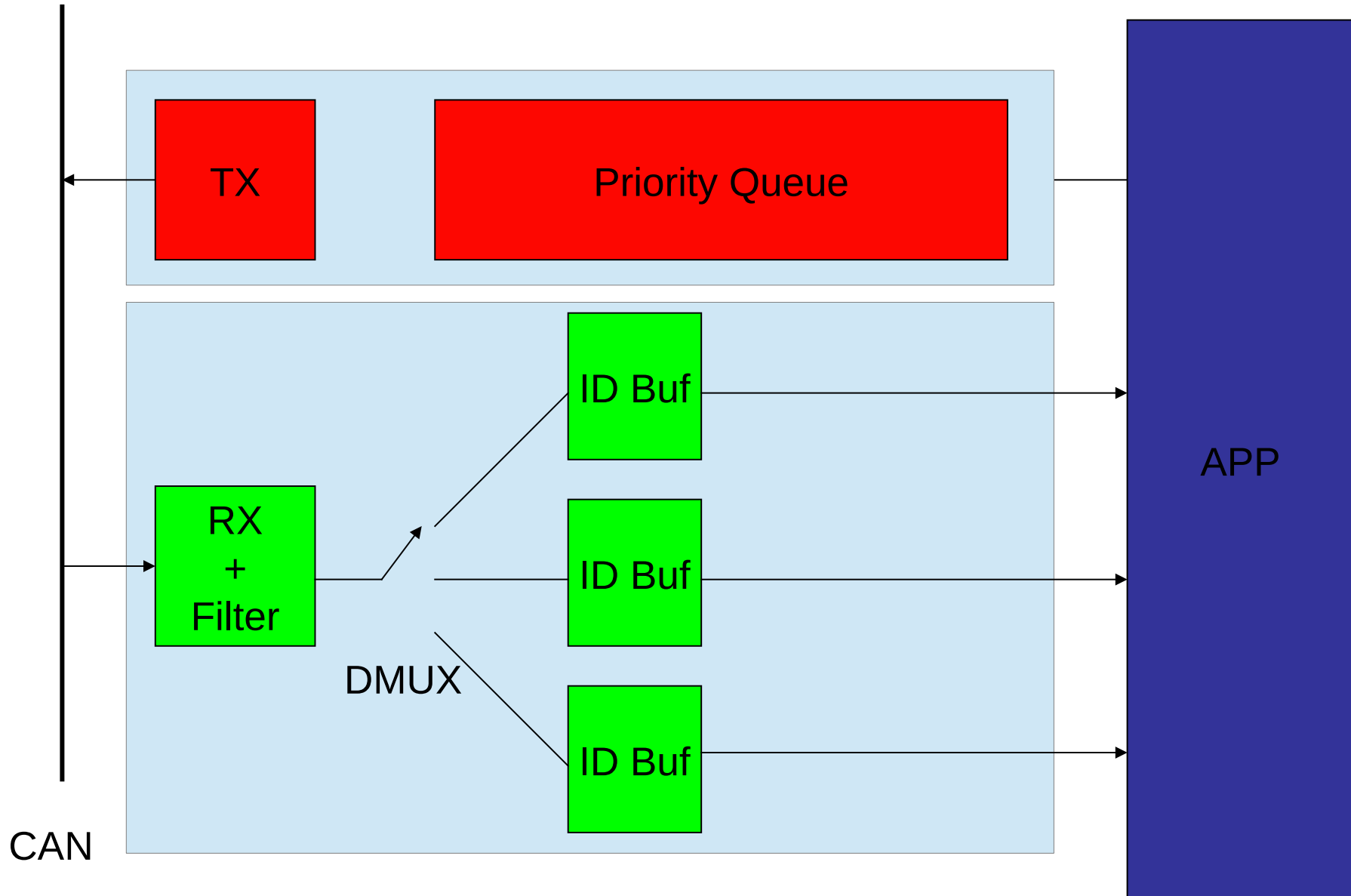
Standard 11 Bit Identifier



15 Bit CRC Generator Polynome = 0b110001011001101

-> Undetected errors will be suppressed by 1 : 10⁻¹⁰

CAN Driver



Lecture Overview

- Introduction, Microcontroller basics
- 32-bit Microcontroller technology
- ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
- Cortex M Microcontroller Instruction Set
- Exception Handling, Interrupts
- Real-time Operating Systems (RTOS)
- Memory Management, protected memory, MPU
- Implementing Calculations, Data Formats, FPU
- I/O, Device drivers
- Application Design
- Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

